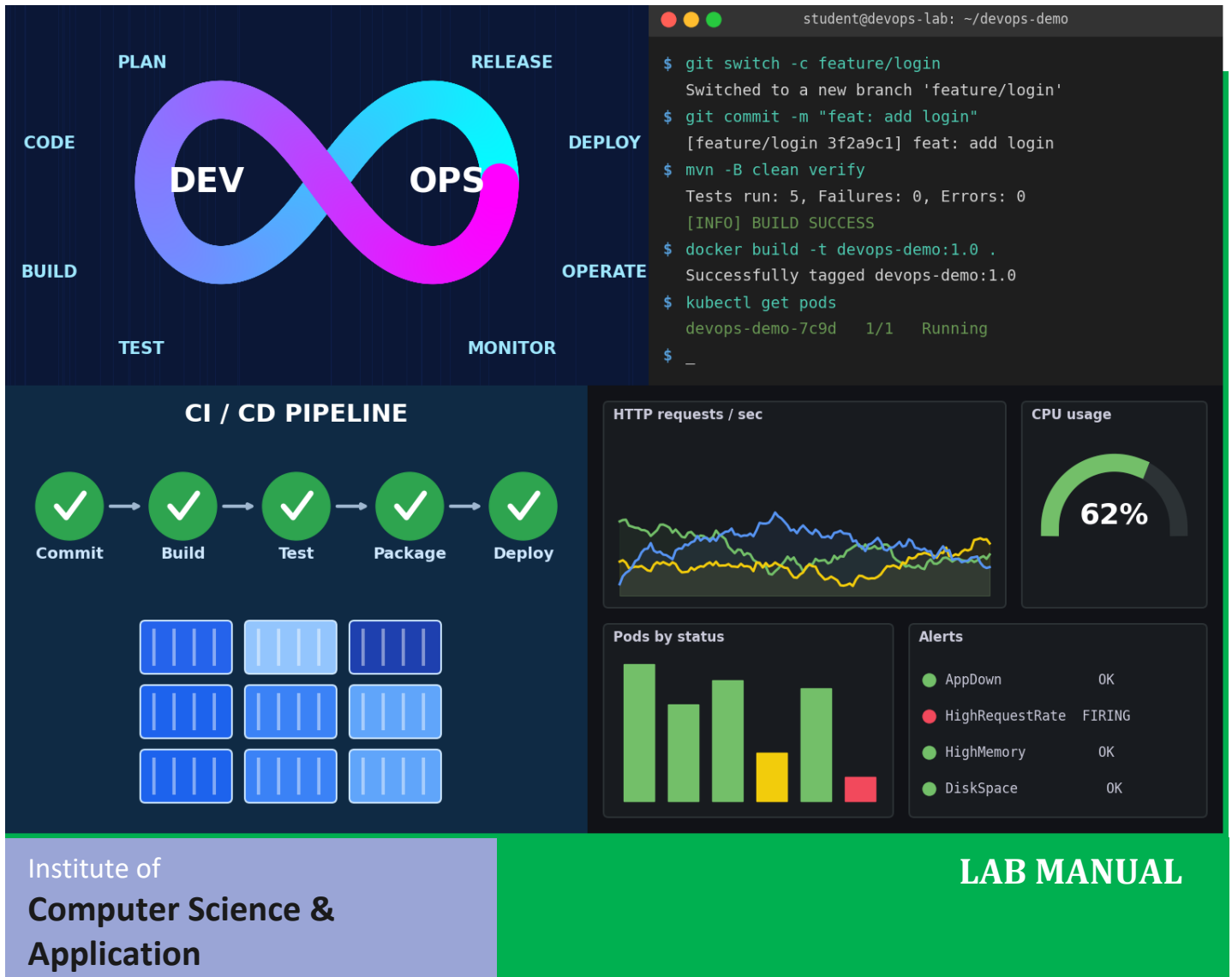




Institute of
**Computer Science &
Application**

LAB MANUAL

Department of Computer Science

DevOps and Agile Foundation

(SSCS4010) | B.Sc. IT/BCA

Name of Student: Shah Rishabh Alpeshbhai

Enrollment No.: 23SS02IT181 **Academic Year:** 2026-27



P P Savani School of Engineering

CERTIFICATE

This is to certify that

Mr. **Shah Rishabh Alpeshbhai** of Semester **7th**

Programme **Bachelor of Science Honours in Information Technology**

Enrollment No. **23SS02IT181** has completed

his/her Term work in the subject of

DEVOPS AND AGILE FOUNDATION

(SSCS4010)

Marks Obtained: _____ out of _____

Signature of Faculty

Date: _____

Signature of Head of Department

Date: _____

CONTENTS

SR NO	NAME OF THE PRACTICAL	PAGE NO	DATE	MARKS	SIGN
1	Git & GitHub basics (clone, commit, branches)	1			
2	Branching strategies & PR workflow (feature branches, rebasing, code review)	6			
3	CI with GitHub Actions (build + unit tests)	11			
4	Jenkins Pipeline (declarative) (simple pipeline + unit tests + artifact publish)	16			
5	Artifact repo & build tools (Maven/Gradle + Nexus)	22			
6	Docker basics & Docker Compose (containerize a sample app)	28			
7	Kubernetes: deploy app, service, scaling (minikube / kind)	34			
8	Monitoring & Logging (Prometheus + Grafana dashboard, basic alert)	40			
9	Mini DevOps project (end-to-end: Git → CI → CD → monitor)	46			

Course Information

Course Code	SSCS4010
Course Name	DevOps and Agile Foundation
Teaching Scheme	Theory: 03 hrs/week Practical: 02 hrs/week Credits: 04
Practical Evaluation	CE: 40 marks ESE: 60 marks
Prerequisite	–

Objective of the Course:

This course introduces the fundamentals of Agile and DevOps practices, emphasizing automation, collaboration, and continuous delivery. Students will gain practical skills with tools like Git, Jenkins, Docker, and Kubernetes to build and manage efficient CI/CD pipelines.

Course Outcomes:

After the completion of the course, students will be able to:

CO	Statement
CO1	Explain Agile values/principles and DevOps culture, benefits and lifecycle.
CO2	Apply Scrum/Kanban practices to plan and run iterative deliveries.
CO3	Use and analyze core DevOps tools: Git workflows, automated builds, CI servers, artifact repo.
CO4	Design and implement CI/CD pipelines, containerized delivery and Infrastructure as Code.
CO5	Integrate monitoring, logging, security scanning, and metrics; deliver a working mini DevOps pipeline/project and reflect on continuous improvement.

Practical – CO Mapping:

Sr. No.	Name of Practical	Hours	CO
1	Git & GitHub basics (clone, commit, branches)	02	CO3
2	Branching strategies & PR workflow (feature branches, rebasing, code review)	04	CO2, CO3
3	CI with GitHub Actions (build + unit tests)	02	CO3, CO4
4	Jenkins Pipeline (declarative) (simple pipeline + unit tests + artifact publish)	04	CO3, CO4
5	Artifact repo & build tools (Maven/Gradle + Nexus)	02	CO3
6	Docker basics & Docker Compose (containerize a sample app)	04	CO4
7	Kubernetes: deploy app, service, scaling (minikube / kind)	02	CO4
8	Monitoring & Logging (Prometheus + Grafana dashboard, basic alert)	02	CO5
9	Mini DevOps project (end-to-end: Git → CI → CD → monitor)	08	CO1 – CO5
	TOTAL	30	

Practical Evaluation Scheme:

- Continuous Evaluation consists of the performance of practical, which will be evaluated out of 10 for each practical. At the end of the semester, the average of all practicals will be converted to 20 marks.
- Internal viva consists of 20 marks.
- Practical performance / quiz / test consists of 30 marks.
- External viva consists of 30 marks.

Rubric for each Practical (10 marks):

Criterion	Marks
Successful execution of all steps (commands / pipeline / deployment working)	4
Observations & screenshots recorded neatly in the manual	2
Understanding shown in Questions & Answers / oral viva	2
Timely submission and completion of the exercise	2

Rules for DevOps Laboratory

1. Keep the working area of the laboratory clean. Shut down or log off your system properly before leaving.
2. Read the allotted practical before coming to the laboratory and keep the required accounts (GitHub, Docker Hub) ready.
3. Stay in the laboratory for the complete lab hours. In case of any doubt, take help of either the teaching assistant or the faculty member.
4. Perform every practical on your own repository and account. Copying another student's repository, commits or screenshots is treated as malpractice.
5. Do not install, uninstall or update any software, driver or system setting other than what the practical requires, without permission.
6. Never store passwords, tokens or private keys in source code or commit them to Git. Use secrets / credentials stores as taught.
7. Stop and remove containers, clusters and services that you started (`docker compose down`, `minikube stop`, `kind delete cluster`) before leaving, so that lab machines stay usable for the next batch.
8. Use of mobile phone is strictly prohibited in the laboratory except when explicitly asked (e.g., for two-factor authentication).
9. Every time, bring the following items for the practical:
 - (a) Laboratory Manual
 - (b) Your GitHub / Docker Hub login credentials (not to be shared)
 - (c) A pen drive or cloud folder for backing up your work (optional)

Lab Safety & Security Rules

For Students

1. Report any hardware fault, electrical issue, smoke or burning smell to the instructor immediately. Do not open the CPU cabinet or touch power cables.
2. Keep the pathway clear by placing bags and other items on the shelves provided. Keep liquids away from keyboards and systems.
3. Do not run unknown scripts, images or binaries downloaded from the internet. Pull container images only from official / verified publishers.
4. Do not attempt to access, scan or attack any machine, network or service other than your own lab setup. Unauthorized access is a punishable offence.
5. Never disable the antivirus, firewall or proxy settings of the lab machines.
6. Lock your screen (Windows + L) whenever you leave your seat, and log out of all web accounts (GitHub, Docker Hub, cloud consoles) at the end of the session.
7. Use strong passwords and enable two-factor authentication on your GitHub account. Use personal access tokens with minimum scope and expiry.
8. Resources such as CPU and RAM are shared — do not run unnecessary containers or scale deployments beyond the limits given in the practical.
9. Leave your work station clean and in good order before leaving the laboratory.
10. Do not leave your assigned laboratory station without permission from the teacher.
11. Learn the location of the fire extinguisher, emergency exit and first aid kit.
12. Follow all instructions given by your teacher.
13. No eating or drinking in the lab at any time!

Lab Setup & Software Requirements

All practicals in this manual build on one common sample application ([devops-demo](#)) so that the work done in each practical feeds into the next and finally into the Mini DevOps Project. Verify the following setup in the first lab session.

Hardware Requirement (per student system):

Sr. No.	Component	Minimum Specification
1	Processor	64-bit, 4 cores with virtualization (VT-x / AMD-V) enabled in BIOS
2	RAM	8 GB (16 GB recommended for Kubernetes + monitoring stack)
3	Free Disk	30 GB
4	Network	Internet access to github.com, Docker Hub, Maven Central

Software Requirement:

Sr. No.	Software / Tool	Version (or later)	Used in Practical
1	Git	2.40	1, 2, 9
2	GitHub account	—	1, 2, 3, 9
3	VS Code (or any IDE) + Git Bash / Terminal	Latest	All
4	JDK (Eclipse Temurin)	17 LTS	3, 4, 5
5	Apache Maven / Gradle	3.9 / 8.x	3, 4, 5
6	Docker Desktop / Docker Engine + Compose v2	24.x	4 – 9
7	Jenkins (run as container)	LTS (JDK 17)	4
8	Sonatype Nexus Repository OSS (container)	3.x	5
9	kubectrl + minikube or kind	1.29+ / 0.22+	7, 9
10	Prometheus, Grafana, Node Exporter (containers)	See Practical 8	8, 9

Verify installation:

Terminal

```
git --version
java -version
mvn -version
docker --version
docker compose version
kubectl version --client
minikube version      # or: kind version
```

Note: On Windows, enable WSL 2 before installing Docker Desktop, and run all shell commands of this manual from Git Bash or a WSL terminal. Commands that differ in PowerShell are shown separately where needed.

Reference Application: devops-demo

`devops-demo` is a small Java 17 web service with no external runtime dependencies. It exposes four HTTP endpoints and is tested with JUnit 5. The faculty will share this code as a starter repository; students must understand it before Practical 1.

Endpoint	Response	Used for
<code>/</code>	Hello from DevOps Demo v<version> (host: <name>)	Functional check, version shown after each deployment
<code>/health</code>	OK	Docker health check, Kubernetes liveness/readiness probes
<code>/add?a=2&b=3</code>	5	Business logic (Calculator class) covered by unit tests
<code>/metrics</code>	Prometheus text format	Monitoring in Practical 8 and 9

Project structure:

Directory tree

```

devops-demo/
├── pom.xml
├── Jenkinsfile
├── Dockerfile
├── docker-compose.yml
├── .github/workflows/ci.yml
├── k8s/
│   ├── deployment.yaml
│   └── service.yaml
└── src/
    ├── main/java/edu/ppsu/devops/
    │   ├── App.java
    │   └── Calculator.java
    └── test/java/edu/ppsu/devops/
        └── CalculatorTest.java
  
```

(Practical 4)
(Practical 6)
(Practical 6 / 8)
(Practical 3 / 9)
(Practical 7 / 9)

pom.xml

pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>edu.ppsu.devops</groupId>
  <artifactId>devops-demo</artifactId>
  <version>1.0.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <junit.version>5.10.2</junit.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>${junit.version}</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
  
```

```

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.5</version>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-jar-plugin</artifactId>
      <version>3.4.1</version>
      <configuration>
        <archive>
          <manifest>
            <mainClass>edu.ppsu.devops.App</mainClass>
          </manifest>
        </archive>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>

```

src/main/java/edu/pps/devops/Calculator.java

```

Calculator.java
package edu.ppsu.devops;

public class Calculator {
    public int add(int a, int b)      { return a + b; }
    public int subtract(int a, int b) { return a - b; }
    public int multiply(int a, int b) { return a * b; }

    public int divide(int a, int b) {
        if (b == 0) {
            throw new ArithmeticException("Division by zero");
        }
        return a / b;
    }
}

```

src/main/java/edu/pps/devops/App.java

```

App.java
package edu.ppsu.devops;

import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpServer;
import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;

public class App {
    static final String VERSION = System.getenv().getOrDefault("APP_VERSION", "1.0");
    static final Map<String, AtomicLong> REQUESTS = new ConcurrentHashMap<>();
    static final long START = System.currentTimeMillis();
    static final String HOST = System.getenv().getOrDefault("HOSTNAME", "localhost");
    static final Calculator CALC = new Calculator();

    public static void main(String[] args) throws IOException {
        int port = Integer.parseInt(System.getenv().getOrDefault("PORT", "8080"));
        HttpServer server = HttpServer.create(new InetSocketAddress(port), 0);
        server.createContext("/", ex -> reply(ex, 200,
            "Hello from DevOps Demo v" + VERSION + " (host: " + HOST + ")\n"));
        server.createContext("/health", ex -> reply(ex, 200, "OK\n"));
        server.createContext("/add", App::add);
        server.createContext("/metrics", App::metrics);
        server.start();
        System.out.println("devops-demo v" + VERSION + " listening on port " + port);
    }
}

```

```

    }

    static void add(HttpExchange ex) throws IOException {
        try {
            Map<String, Integer> q = new ConcurrentHashMap<>();
            String query = ex.getRequestURI().getQuery();
            for (String p : query.split("&")) {
                String[] kv = p.split("=");
                q.put(kv[0], Integer.parseInt(kv[1]));
            }
            reply(ex, 200, CALC.add(q.get("a"), q.get("b")) + "\n");
        } catch (Exception e) {
            reply(ex, 400, "Usage: /add?a=2&b=3\n");
        }
    }

    static void metrics(HttpExchange ex) throws IOException {
        StringBuilder sb = new StringBuilder();
        sb.append("# HELP app_info Application version info\n");
        sb.append("# TYPE app_info gauge\n");
        sb.append("app_info{version=\"\"}.append(VERSION).append(\"\"} 1\n");
        sb.append("# HELP http_requests_total Total HTTP requests by path\n");
        sb.append("# TYPE http_requests_total counter\n");
        REQUESTS.forEach((path, n) -> sb.append("http_requests_total{path=\"\"}
            .append(path).append(\"\"} ").append(n.get()).append('\n'));
        Runtime rt = Runtime.getRuntime();
        sb.append("# HELP jvm_memory_used_bytes JVM heap memory in use\n");
        sb.append("# TYPE jvm_memory_used_bytes gauge\n");
        sb.append("jvm_memory_used_bytes ").append(rt.totalMemory() - rt.freeMemory()).append('\n');
        sb.append("# HELP app_uptime_seconds Seconds since the application started\n");
        sb.append("# TYPE app_uptime_seconds gauge\n");
        sb.append("app_uptime_seconds ").append((System.currentTimeMillis() - START) / 1000).append('\n');
        reply(ex, 200, sb.toString());
    }

    static void reply(HttpExchange ex, int status, String body) throws IOException {
        String path = ex.getHttpContext().getPath();
        if (!path.equals("/metrics")) {
            REQUESTS.computeIfAbsent(path, k -> new AtomicLong()).incrementAndGet();
            System.out.println(ex.getRequestMethod() + " " + ex.getRequestURI() + " -> " + status);
        }
        byte[] bytes = body.getBytes(StandardCharsets.UTF_8);
        ex.getResponseHeaders().set("Content-Type", "text/plain; charset=utf-8");
        ex.sendResponseHeaders(status, bytes.length);
        try (OutputStream os = ex.getResponseBody()) {
            os.write(bytes);
        }
    }
}

```

src/test/java/edu/ppsu/devops/CalculatorTest.java

```

CalculatorTest.java
package edu.ppsu.devops;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

import org.junit.jupiter.api.Test;

class CalculatorTest {
    private final Calculator calc = new Calculator();

    @Test
    void addsTwoNumbers() { assertEquals(5, calc.add(2, 3)); }

    @Test
    void subtractsTwoNumbers() { assertEquals(1, calc.subtract(3, 2)); }

    @Test
    void multipliesTwoNumbers() { assertEquals(6, calc.multiply(2, 3)); }
}

```

```
@Test
void dividesTwoNumbers() { assertEquals(2, calc.divide(6, 3)); }

@Test
void divideByZeroThrows() {
    assertThrows(ArithmeticException.class, () -> calc.divide(1, 0));
}
}
```

Build and run locally:

Terminal

<code>mvn -B clean verify</code>	<code># compile + run 5 unit tests</code>
<code>java -jar target/devops-demo-1.0.0-SNAPSHOT.jar</code>	
<code>curl http://localhost:8080/</code>	<code># Hello from DevOps Demo v1.0 (host: localhost)</code>
<code>curl "http://localhost:8080/add?a=2&b=3"</code>	<code># 5</code>

Practical 1

Git & GitHub Basics (clone, commit, branches)

Aim: To install and configure Git, create and clone a repository on GitHub, record changes using commits, and create, switch and merge branches.

Course Outcome: CO3 | **Duration:** 02 Hours

Objectives:

- Understand the purpose of a Version Control System (VCS) and the Git data model.
- Use the basic Git workflow: working directory → staging area → local repository → remote repository.
- Create branches, switch between them and merge them back.

Software Required:

Sr. No.	Software / Tool	Specification	Purpose
1	Git	2.40 or later	Distributed version control
2	GitHub account	Free account	Remote repository hosting
3	VS Code / Git Bash	Latest	Editor and terminal

Theory:

A Version Control System (VCS) records changes to files over time so that any earlier version can be recalled, compared or restored. It allows many developers to work on the same code base in parallel and keeps a complete audit trail of who changed what, when and why.

Git is a distributed VCS created by Linus Torvalds in 2005. Unlike centralized systems (e.g., SVN), every developer has a full copy of the repository including its entire history, so most operations are local and fast. GitHub is a cloud service that hosts Git repositories and adds collaboration features such as pull requests, issues, Actions (CI/CD) and project boards.

The three areas of Git

Area	Description	Command to move forward
Working Directory	Files you are currently editing on disk	<code>git add <file></code>
Staging Area (Index)	Snapshot of changes selected for the next commit	<code>git commit -m "msg"</code>
Local Repository (.git)	Committed history stored on your machine	<code>git push</code>
Remote Repository	Shared copy on GitHub (usually called <code>origin</code>)	<code>git pull</code> / <code>git fetch</code> brings changes back

Commits and branches

A commit is a snapshot of the entire project identified by a SHA-1 hash (e.g., `3f2a9c1`). Each commit stores the author, date, message and a pointer to its parent commit, forming a history graph. A branch is simply a movable pointer to a commit. `HEAD` points to the branch you are currently on. Creating a branch is cheap, which lets developers isolate new features or fixes without disturbing the stable `main` branch.

Frequently used commands

Command	Purpose
<code>git config --global user.name "Name"</code>	Set identity recorded in every commit
<code>git init / git clone <url></code>	Create a new repository / copy an existing one
<code>git status</code>	Show modified, staged and untracked files
<code>git add <file> / git add .</code>	Stage changes
<code>git commit -m "message"</code>	Save staged changes as a new commit
<code>git log --oneline --graph --all</code>	View commit history as a graph
<code>git diff / git diff --staged</code>	Compare working tree / staged changes
<code>git branch / git switch -c <name></code>	List branches / create and switch to a new branch
<code>git merge <branch></code>	Merge another branch into the current branch
<code>git push -u origin <branch></code>	Upload a branch and set upstream tracking
<code>git pull</code>	Fetch and merge changes from the remote
<code>git restore <file></code>	Discard unstaged changes in a file

Procedure:

Step 1: Configure Git (one time on each machine).

Terminal

```
git config --global user.name "Your Full Name"
git config --global user.email "your-email@example.com"
git config --global init.defaultBranch main
git config --list
```

Step 2: Create a repository on GitHub. Log in to github.com → New repository → name it `devops-demo` → select Public → tick Add a README file → Create repository. Copy the HTTPS URL from the green Code button.

Step 3: Clone the repository and inspect it.

Terminal

```
git clone https://github.com/<your-username>/devops-demo.git
cd devops-demo
ls -a          # note the hidden .git folder
git status
git log --oneline
```

Step 4: Add the starter application code shared by the faculty (see Reference Application) into the cloned folder, then create a `.gitignore` file so that build outputs are not committed.

.gitignore

```
# .gitignore
target/
build/
*.class
.idea/
.vscode/
```

Step 5: Stage and commit the changes.

Terminal

```
git status          # files shown in red = untracked
git add .
git status          # files shown in green = staged
git commit -m "Add devops-demo starter application"
git log --oneline
```

Step 6: Push the commit to GitHub and refresh the repository page in the browser to verify. When prompted for a password, use a Personal Access Token (GitHub → Settings → Developer settings → Personal access tokens) or sign in through the browser window opened by Git Credential Manager.

Terminal

```
git push origin main
```

Step 7: Create a branch and make a change on it.

Terminal

```
git switch -c feature/greeting      # create + switch
# edit App.java: change the "/" message to include your name
git diff
git add src/main/java/edu/ppsu/devops/App.java
git commit -m "Personalise greeting message"
git branch                          # * marks the current branch
git log --oneline --graph --all
```

Step 8: Merge the branch into main and push.

Terminal

```
git switch main
git merge feature/greeting          # fast-forward merge
git push origin main
git branch -d feature/greeting      # delete merged branch
```

Step 9: Undo practice (local only).

Terminal

```
echo "temp" >> README.md
git restore README.md               # discard unstaged change
git log --oneline -5
```

Expected Output:

Sample output

```
$ git log --oneline --graph --all
* 7b1e4d2 (HEAD -> main, origin/main) Personalise greeting message
* 3f2a9c1 Add devops-demo starter application
* 91c0a5e Initial commit
```

Exercise for Students:

1. Create a branch `docs/readme`, add build instructions to `README.md`, commit and merge it into `main`.
2. Make three commits and use `git log --oneline` and `git show <hash>` to inspect each of them.
3. Record the screenshots of: `git log --oneline --graph --all`, and the GitHub repository page showing your commits.

Observations / Output:

Paste or draw the screenshots / terminal output requested above (attach printouts if required).

Git was installed and configured, the devops-demo repository was cloned from GitHub, a docs/readme branch was created, three commits were made and the branch was merged into main.

```
$ git --version
git version 2.45.1.windows.1
$ git config --global user.name "Shah_Rishabh"
$ git clone https://github.com/<username>/devops-demo.git
Cloning into 'devops-demo'...
remote: Enumerating objects: 14, done.
Receiving objects: 100% (14/14), done.
$ git checkout -b docs/readme
Switched to a new branch 'docs/readme'
```

```
$ git add README.md
$ git commit -m "docs: add build instructions to README"
[docs/readme 8c2d1f4] docs: add build instructions to README
1 file changed, 12 insertions(+)
$ git commit -am "docs: add prerequisites section"
[docs/readme 2b6c3d1] docs: add prerequisites section
$ git commit -am "docs: add run instructions"
[docs/readme e5a7b90] docs: add run instructions
$ git checkout main
Switched to branch 'main'
$ git merge docs/readme
Updating 7b1e4d2..e5a7b90
Fast-forward
 README.md | 27 +++++++++++++++++++++++++++++++++++++
 1 file changed, 27 insertions(+)
$ git push origin main
To https://github.com/<username>/devops-demo.git
 7b1e4d2..e5a7b90  main -> main
$ git log --oneline --graph --all
* e5a7b90 (HEAD -> main, origin/main, docs/readme) docs: add run instructions
* 2b6c3d1 docs: add prerequisites section
* 8c2d1f4 docs: add build instructions to README
* 7b1e4d2 Personalise greeting message
* 3f2a9c1 Add devops-demo starter application
* 91c0a5e Initial commit
```

Observation: git status showed the README change as modified (working directory), then as staged after git add, and as committed after git commit. Because main had no new commits while docs/readme was being developed, the merge was a fast-forward and no merge commit was created. The GitHub repository page shows all six commits on main.

Result / Conclusion:

Write what you learned about the Git workflow and branching.

The Git workflow (working directory → staging area → local repository → remote repository) was practised successfully. A repository was cloned, changes were recorded with commits, a branch was created and merged back into main, and the result was pushed to GitHub. Branching in Git is lightweight and lets work be done in isolation without disturbing the stable main branch.

Questions & Answers

1. What is the difference between a centralized and a distributed version control system?

Ans: A centralized VCS (e.g., SVN) keeps the entire history on one central server; developers only hold a working copy and need network access to commit or view history. A distributed VCS (e.g., Git) gives every developer a full copy of the repository including its complete history, so commits, branching and log can be done offline, and there is no single point of failure.

2. Explain the working directory, staging area and repository in Git.

Ans: Working directory: the folder on disk where files are edited. Staging area (index): a holding area where selected changes are prepared with `git add` before committing. Repository (`.git`): the database that stores all committed snapshots and history. Flow: edit → `git add` → `git commit`.

3. What is the difference between `git fetch` and `git pull`?

Ans: `git fetch` downloads new commits and branches from the remote into remote-tracking branches (e.g., `origin/main`) without changing your working files. `git pull` = `git fetch` + `git merge` (or `rebase`); it downloads and immediately integrates the remote changes into the current branch.

4. What is a branch in Git? Why is creating a branch cheap?

Ans: A branch is a movable pointer to a commit that represents an independent line of development. Creating a branch is cheap because Git only creates a small file holding a 41-byte commit hash; no files are copied.

5. What is the purpose of the `.gitignore` file?

Ans: The `.gitignore` file lists files and folders (e.g., `target/`, `*.class`, `.idea/`, `secrets`) that Git should not track, so build outputs, temporary files and credentials are not committed to the repository.

Teacher's Signature	Teacher's Note	Marks (out of 10)

Practical 2

Branching Strategies & Pull Request Workflow (feature branches, rebasing, code review)

Aim: To practise a feature-branch workflow on GitHub using pull requests, perform rebasing and conflict resolution, carry out a peer code review, and protect the main branch.

Course Outcome: CO2, CO3 | **Duration:** 04 Hours

Objectives:

- Compare popular branching strategies: GitHub Flow, Git Flow and Trunk-Based Development.
- Understand merge vs. rebase and resolve merge conflicts.
- Raise, review and merge Pull Requests (PRs) as a team; apply branch protection rules.

Software Required:

Sr. No.	Software / Tool	Specification	Purpose
1	Git	2.40 or later	Local branching, rebasing
2	GitHub	Repository from Practical 1	Pull requests, reviews, branch protection
3	VS Code	Latest	Conflict resolution editor

Theory:

Branching strategies

Strategy	Key idea	Best suited for
GitHub Flow	Single long-lived <code>main</code> ; short-lived feature branches merged through PRs; <code>main</code> is always deployable.	Web apps with continuous deployment
Git Flow	Long-lived <code>main</code> and <code>develop</code> , plus <code>feature/*</code> , <code>release/*</code> and <code>hotfix/*</code> branches.	Products with scheduled, versioned releases
Trunk-Based Development	Everyone commits small changes to trunk (<code>main</code>) at least daily; feature flags hide unfinished work.	High-performing teams practising CI/CD

Merge vs. Rebase

Merge joins two histories by creating a merge commit that has two parents; the original branch history is preserved exactly. **Rebase** moves (re-applies) the commits of your branch on top of the latest commit of another branch, producing a linear history. Rebasing rewrites commit hashes, so the golden rule is: never rebase commits that others have already pulled — rebase only your own local / feature branch.

Pull Requests and Code Review

A Pull Request (PR) asks maintainers to review and merge changes from one branch into another. It shows the diff, allows line-by-line comments, runs automated checks (CI), and records approvals. Good reviews check correctness, readability, tests, security and adherence to coding standards. Branch protection rules enforce the process, e.g., require a PR before merging, require 1 approval, require status checks to pass and block force pushes.

Merge options on GitHub

- **Create a merge commit** – keeps all commits plus a merge commit.
- **Squash and merge** – combines all PR commits into one commit on `main`.
- **Rebase and merge** – re-applies each commit on `main` without a merge commit.

Conventional commit messages

Teams often follow the format `type(scope): summary`, e.g., `feat(api): add /subtract endpoint`, `fix: handle divide by zero`, `docs: update README`, `test: add multiply tests`. This keeps history readable and enables automatic changelogs.

Procedure:

Work in pairs (Student A = repository owner, Student B = collaborator). Student A adds Student B under Settings → Collaborators → Add people.

Part A – Feature branch and Pull Request

Step 1: Create a feature branch (Student B).

```
Terminal
git clone https://github.com/<owner>/devops-demo.git
cd devops-demo
git switch -c feature/subtract-endpoint
```

Step 2: Implement the change. Add a `/subtract` endpoint in `App.java` (similar to `/add`) and a new test in `CalculatorTest.java`. Commit in small logical steps.

```
Terminal
git add src/main/java/edu/ppsu/devops/App.java
git commit -m "feat(api): add /subtract endpoint"
git add src/test/java/edu/ppsu/devops/CalculatorTest.java
git commit -m "test: add subtract test case"
mvn -B clean verify # make sure tests pass before pushing
git push -u origin feature/subtract-endpoint
```

Step 3: Open a Pull Request. On GitHub click Compare & pull request → base: `main`, compare: `feature/subtract-endpoint` → write a description (what, why, how tested) → add Student A as Reviewer → Create pull request.

Part B – Code review

Step 4: Review (Student A). Open the PR → Files changed → click + on a line to add a comment (e.g., "Please validate missing query parameters") → Review changes → select Request changes → Submit review.

Step 5: Address the review (Student B). Make the fix locally, commit and push to the same branch. The PR updates automatically. Reply to the comment and click Resolve conversation. Student A then approves the PR.

Part C – Rebase and conflict resolution

Step 6: Create a conflict deliberately. Student A edits the greeting line in `App.java` directly on `main` and pushes. Student B edits the same line on the feature branch.

Step 7: Rebase the feature branch on the latest main (Student B).

```
Terminal
git fetch origin
git rebase origin/main
# CONFLICT (content): Merge conflict in src/main/java/edu/ppsu/devops/App.java
# Open the file, keep the correct text and remove the markers:
# <<<<<< HEAD ... ===== ... >>>>>> feature commit
git add src/main/java/edu/ppsu/devops/App.java
```

```
git rebase --continue
git log --oneline --graph --all      # history is now linear
git push --force-with-lease          # needed because hashes changed
```

Step 8: Squash commits with interactive rebase (optional).

```
Terminal
git rebase -i HEAD~2
# in the editor change the second line from "pick" to "squash", save & close
git push --force-with-lease
```

Note: `--force-with-lease` refuses to overwrite the remote branch if someone else pushed to it meanwhile; it is the safe alternative to `--force`.

Step 9: Merge the PR. Student A selects Squash and merge → Confirm. Delete the branch from GitHub when prompted. Both students run `git switch main && git pull`.

Part D – Protect the main branch

Step 10: Add a branch protection rule (Student A). Repository Settings → Branches → Add branch protection rule (or Rules → Rulesets in newer GitHub UI) → branch name pattern `main` → enable Require a pull request before merging with Require approvals = 1, and Do not allow bypassing the above settings → Create / Save.

Step 11: Verify the rule. Try `git push origin main` with a direct commit — the push must be rejected with `protected branch hook declined`.

Expected Output:

```
Sample output
* a41c7e0 (HEAD -> main, origin/main) feat(api): add /subtract endpoint (#1)
* 5d2b9f8 Update greeting text
* 7b1e4d2 Personalise greeting message

remote: error: GH006: Protected branch update failed for refs/heads/main.
```

Observation Table:

Sr. No.	Branch name	PR No.	Reviewer	Review status	Merge method
1	feature/subtract-endpoint	#1	Classmate (peer)	Approved	Squash and merge
2	feature/greeting-text	#2	Classmate (peer)	Changes requested, then approved	Rebase and merge
3	docs/codeowners	#3	Classmate (peer)	Approved	Create a merge commit

Exercise for Students:

1. Draw the commit graph before and after `git rebase origin/main` for your feature branch.
2. Create a second PR that is merged using Create a merge commit and compare the history with the squash merge.
3. Add a `CODEOWNERS` file (`* @<owner-username>`) and observe how reviewers are requested automatically.

Observations / Output:

Paste or draw the screenshots / terminal output requested above (attach printouts if required).

A feature branch was created, pushed and merged through a Pull Request after peer review; a rebase conflict was resolved; and branch protection was enabled on main.

```
$ git checkout -b feature/subtract-endpoint
$ git commit -m "feat(api): add /subtract endpoint"
$ git push -u origin feature/subtract-endpoint
remote: Create a pull request for 'feature/subtract-endpoint' on GitHub
(PR #1 reviewed by peer -> Approved -> Squash and merge)
$ git checkout main && git pull
$ git log --oneline --graph --all
* a41c7e0 (HEAD -> main, origin/main) feat(api): add /subtract endpoint (#1)
* 5d2b9f8 Update greeting text
* 7b1e4d2 Personalise greeting message

$ git fetch origin && git rebase origin/main
CONFLICT (content): Merge conflict in src/main/java/edu/ppsu/devops/App.java
$ (edit file, remove <<<<<< ===== >>>>>> markers)
$ git add src/main/java/edu/ppsu/devops/App.java
$ git rebase --continue
Successfully rebased and updated refs/heads/feature/greeting-text.
$ git push --force-with-lease

$ git push origin main
remote: error: GH006: Protected branch update failed for refs/heads/main.
```

Observation: after rebasing, the history of the feature branch became a straight line on top of origin/main, whereas the merge-commit PR (#3) added an extra merge commit with two parents. A direct push to main was rejected once 'Require a pull request before merging' was enabled.

Result / Conclusion:

Compare merge and rebase based on the history graphs you obtained.

Merge keeps the exact history and adds a merge commit, while rebase rewrites the feature commits on top of the latest main and gives a clean, linear history. A PR-based workflow with peer review and branch protection prevents unreviewed code from reaching main and improves code quality. Rebase should only be used on local/private branches, never on shared branches.

Questions & Answers

1. Compare GitHub Flow, Git Flow and Trunk-Based Development.

Ans: GitHub Flow: a single long-lived main branch plus short-lived feature branches merged through PRs; simple and suited to continuous deployment. Git Flow: uses main, develop, feature, release and hotfix branches; suited to scheduled releases but heavier. Trunk-Based Development: everyone commits small changes to one trunk (main) frequently, using feature flags; best for mature CI/CD teams.

2. What is the difference between `git merge` and `git rebase`? When should rebase be avoided?

Ans: `git merge` joins two branches and creates a merge commit, preserving the full history. `git rebase` replays your commits on top of another branch, producing a linear history but rewriting commit hashes. Rebase should be avoided on public/shared branches that others have already pulled.

3. What is a merge conflict and how is it resolved?

Ans: A merge conflict occurs when two branches change the same lines of a file (or one deletes a file the other edits) and Git cannot combine them automatically. It is resolved by opening the file, choosing the correct content between the conflict markers, removing the markers, then running `git add` and `git commit` (or `git rebase --continue`).

4. List any four points a reviewer should check during a code review.

Ans: (1) Code correctness and logic; (2) Readability and coding standards/naming; (3) Presence and quality of unit tests; (4) Security issues such as hard-coded secrets and input validation (also: performance, small PR size, CI passing).

5. Why is `--force-with-lease` preferred over `--force`?

Ans: `--force` overwrites the remote branch blindly and can erase commits pushed by others. `--force-with-lease` only pushes if the remote branch is still at the commit you last fetched, so it refuses to overwrite someone else's new work, making it much safer after a rebase.

Teacher's Signature	Teacher's Note	Marks (out of 10)

Practical 3

Continuous Integration with GitHub Actions (build + unit tests)

Aim: To create a GitHub Actions workflow that automatically builds the `devops-demo` application and runs its unit tests on every push and pull request.

Course Outcome: CO3, CO4 | **Duration:** 02 Hours

Objectives:

- Understand Continuous Integration (CI) and the structure of a GitHub Actions workflow.
- Write a YAML workflow to build and test a Maven project and publish test reports.
- Observe how CI blocks a pull request when a test fails.

Software Required:

Sr. No.	Software / Tool	Specification	Purpose
1	GitHub Actions	GitHub-hosted runner <code>ubuntu-latest</code>	CI execution
2	JDK / Maven	17 / 3.9 (provided on the runner)	Build and test
3	Repository	<code>devops-demo</code> from Practical 1-2	Source code

Theory:

Continuous Integration (CI) is the practice of merging every developer's work into a shared mainline frequently (at least daily) and verifying each merge with an automated build and test run. CI detects integration problems early, keeps the main branch always in a working state and gives developers fast feedback.

GitHub Actions is the CI/CD service built into GitHub. Workflows are written in YAML and stored in the `.github/workflows/` folder of the repository (pipeline as code).

Term	Meaning
Workflow	An automated process defined in a YAML file; contains one or more jobs.
Event (on)	What triggers the workflow – <code>push</code> , <code>pull_request</code> , <code>schedule</code> , <code>workflow_dispatch</code> (manual), etc.
Job	A set of steps that run on the same runner. Jobs run in parallel unless <code>needs:</code> is specified.
Step	A single shell command (<code>run:</code>) or a reusable action (<code>uses:</code>).
Action	A reusable unit, e.g., <code>actions/checkout</code> , <code>actions/setup-java</code> .
Runner	The machine that executes the job (GitHub-hosted or self-hosted).
Artifact	Files produced by a job and stored with the run (e.g., test reports, JARs).

Maven build lifecycle used in CI

`mvn -B clean verify` runs the phases `clean` → `validate` → `compile` → `test` → `package` → `verify`. The `-B` (batch) flag gives non-interactive, log-friendly output. If any unit test fails, Maven exits with a non-zero code and the CI job is marked as failed.

Procedure:

Step 1: Create the workflow file in your repository at `.github/workflows/ci.yml` (create the folders if they do not exist).

```
.github/workflows/ci.yml
name: CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
  workflow_dispatch:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout source
        uses: actions/checkout@v4

      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '17'
          cache: maven

      - name: Build and run unit tests
        run: mvn -B clean verify

      - name: Upload test reports
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: surefire-reports
          path: target/surefire-reports/

      - name: Upload JAR
        uses: actions/upload-artifact@v4
        with:
          name: devops-demo-jar
          path: target/*.jar
```

Step 2: Commit and push the workflow.

```
Terminal
git switch -c ci/github-actions
git add .github/workflows/ci.yml
git commit -m "ci: add GitHub Actions build and test workflow"
git push -u origin ci/github-actions
```

Step 3: Open a Pull Request from `ci/github-actions` to `main`. Observe the check CI / build running at the bottom of the PR. Click Details to view the live log of every step.

Step 4: Inspect the run. Go to the Actions tab → select the run → expand Build and run unit tests and note `Tests run: 5, Failures: 0`. Download the `surefire-reports` and `devops-demo-jar` artifacts from the run summary page.

Step 5: Break the build intentionally. Change an expected value in `CalculatorTest` (e.g., `assertEquals(6, calc.add(2, 3))`), commit and push. Observe the red X on the PR and the failure message in the log. Fix the test and push again to turn the check green.

Step 6: Make the check mandatory. In Settings → Branches (rule for `main`) enable Require status checks to pass before merging and select `build`. A PR with failing tests can no longer be merged.

Step 7: Add a status badge to `README.md`:**README.md**

```

```

Step 8: Merge the PR once the check is green and confirm a new workflow run starts on `main` (push event).

Expected Output:**Actions log (excerpt)**

```
[INFO] -----
[INFO]  T E S T S
[INFO] -----
[INFO] Running edu.ppsu.devops.CalculatorTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Observation Table:

Run No.	Trigger event	Commit / PR	Tests run / failed	Duration	Status
1	push (feature branch)	a3f9c21	5 / 0	41 s	Success
2	pull_request	PR #4 (test deliberately broken)	5 / 1	38 s	Failure
3	push (main, after fix)	e7b2d05	5 / 0	43 s	Success

Exercise for Students:

1. Add a build matrix so the job runs on Java 17 and 21 (`strategy: matrix: java: [17, 21]`).
2. Add a schedule trigger to run the workflow every night at 2:00 AM UTC (`cron: "0 2 * * *"`).
3. Record screenshots of a failed run and a successful run on the same pull request.

Observations / Output:

Paste or draw the screenshots / terminal output requested above (attach printouts if required).

The workflow `.github/workflows/ci.yml` ran automatically on every push and pull request.

```
[INFO] -----
[INFO]  T E S T S
[INFO] -----
[INFO] Running edu.ppsu.devops.CalculatorTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS

--- Run #2 (test broken on purpose) ---
[ERROR] Tests run: 5, Failures: 1, Errors: 0, Skipped: 0
[ERROR] CalculatorTest.addsTwoNumbers: expected: <5> but was: <6>
[INFO] BUILD FAILURE
Process completed with exit code 1. (PR check: X ci / build-test)
```

Observation: in run #2 the red cross appeared on the pull request and the Merge button was blocked; the test report artifact was still uploaded because of `if: always()`. After fixing the test, run #3 turned green and the PR could be merged.

Result / Conclusion:

Explain how CI gives fast feedback using your observations of the passing and failing runs.

GitHub Actions built the project and executed all five unit tests automatically on every push and pull request. The failed run showed the defect within about 40 seconds, before the code reached main, and the next successful run confirmed the fix. This shows that Continuous Integration gives fast feedback, catches bugs early and keeps main always in a working state.

Questions & Answers

1. What is Continuous Integration? List its benefits.

Ans: Continuous Integration is the practice of frequently merging developers' code into a shared repository, where each change is automatically built and tested. Benefits: early bug detection, fast feedback, less integration conflict, always-working main branch, and faster, more reliable releases.

2. Explain workflow, job, step, action and runner in GitHub Actions.

Ans: Workflow: an automated process defined in a YAML file in `.github/workflows`. Job: a group of steps that runs on one runner. Step: a single task (a shell command or an action). Action: a reusable unit of code (e.g., `actions/checkout`). Runner: the virtual machine (e.g., `ubuntu-latest`) that executes the jobs.

3. What is the use of `actions/checkout` and `actions/setup-java`?

Ans: `actions/checkout` downloads the repository code onto the runner so the build can access it. `actions/setup-java` installs the required JDK (e.g., Temurin 17) and can cache Maven dependencies for faster builds.

4. Why do we use `if: always()` for uploading test reports?

Ans: `if: always()` makes a step run even when an earlier step (such as the tests) failed. Without it the report upload would be skipped on failure, and the failing test reports - the ones most needed for debugging - would be lost.

5. How can CI be made mandatory before merging a pull request?

Ans: By enabling branch protection on main (Settings → Branches / Rulesets) and selecting 'Require status checks to pass before merging' with the CI job (e.g., `build-test`) chosen as a required check; the PR then cannot be merged until the workflow is green.

Teacher's Signature	Teacher's Note	Marks (out of 10)

Practical 4

Jenkins Pipeline (Declarative) – simple pipeline + unit tests + artifact publish

Aim: To install Jenkins, create a declarative pipeline (Jenkinsfile) for the `devops-demo` application that checks out code, builds it, runs unit tests, publishes test results and archives the generated JAR as a build artifact.

Course Outcome: CO3, CO4 | **Duration:** 04 Hours

Objectives:

- Run Jenkins as a Docker container and complete its initial setup.
- Understand the structure of a declarative pipeline: agent, tools, stages, steps, post.
- Publish JUnit test reports and archive build artifacts; trigger builds automatically from SCM.

Software Required:

Sr. No.	Software / Tool	Specification	Purpose
1	Docker	24.x	Run Jenkins container
2	Jenkins	<code>jenkins/jenkins:lts-jdk17</code>	CI server
3	Jenkins plugins	Suggested plugins (Pipeline, Git, JUnit, Stage View)	Pipeline features
4	Maven	3.9.x (auto-installed by Jenkins)	Build tool

Theory:

Jenkins is an open-source automation server written in Java and one of the most widely used CI/CD tools. Its functionality is extended through more than 1,800 plugins. A Jenkins installation has a controller (web UI, scheduling, configuration) and can use additional agents (nodes) that execute builds.

Pipeline as Code: a Jenkins pipeline is defined in a text file called `Jenkinsfile` that is stored with the source code in Git. This makes the pipeline versioned, reviewable and repeatable. Jenkins supports two syntaxes – Scripted (Groovy code, very flexible) and Declarative (structured, easier to read, recommended for most cases).

Directive	Purpose
<code>pipeline { }</code>	Root block of every declarative pipeline
<code>agent</code>	Where the pipeline runs: <code>any</code> , a label, or a Docker image
<code>tools</code>	Auto-installs and adds tools (Maven, JDK) to PATH
<code>options</code>	Pipeline options such as timeout and build history retention
<code>environment</code>	Environment variables available to all stages
<code>stages / stage / steps</code>	Logical phases (Build, Test ...) and the commands inside them
<code>post</code>	Actions after a stage / pipeline: <code>always</code> , <code>success</code> , <code>failure</code> , <code>unstable</code>
<code>junit</code>	Publishes JUnit XML results – trend graph & test failure details
<code>archiveArtifacts</code>	Stores files (e.g., JAR) with the build so they can be downloaded later

Procedure:

Part A – Install Jenkins

Step 1: Start Jenkins in Docker.

Terminal

```
docker volume create jenkins_home
docker run -d --name jenkins --restart unless-stopped \
  -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  jenkins/jenkins:lts-jdk17

# read the initial admin password
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Step 2: Unlock Jenkins. Open <http://localhost:8080>, paste the password → Install suggested plugins → create the first admin user → keep the Jenkins URL → Start using Jenkins.

Step 3: Configure Maven. Manage Jenkins → Tools → Maven installations → Add Maven → Name: **Maven3** → tick Install automatically (choose a 3.9.x version) → Save.

Part B – Write the Jenkinsfile

Step 4: Add a **Jenkinsfile** at the root of the repository and push it to GitHub.

Jenkinsfile

```
pipeline {
  agent any

  tools {
    maven 'Maven3'           // name configured in Manage Jenkins > Tools
  }

  options {
    buildDiscarder(logRotator(numToKeepStr: '10'))
    timeout(time: 15, unit: 'MINUTES')
  }

  environment {
    APP_NAME = 'devops-demo'
  }

  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }
    stage('Build') {
      steps {
        sh 'mvn -B -DskipTests clean compile'
      }
    }
    stage('Unit Test') {
      steps {
        sh 'mvn -B test'
      }
      post {
        always {
          junit 'target/surefire-reports/*.xml'
        }
      }
    }
    stage('Package') {
      steps {
        sh 'mvn -B -DskipTests package'
      }
    }
    stage('Publish Artifact') {
      steps {
```

```

        archiveArtifacts artifacts: 'target/*.jar', fingerprint: true
    }
}
}

post {
    success {
        echo "Build #${env.BUILD_NUMBER} of ${APP_NAME} succeeded"
    }
    failure {
        echo "Build #${env.BUILD_NUMBER} failed - check Console Output"
    }
}
}
}

```

Terminal

```

git add Jenkinsfile
git commit -m "ci: add declarative Jenkins pipeline"
git push origin main

```

Part C – Create and run the pipeline job

Step 5: Create the job. New Item → name `devops-demo-pipeline` → select Pipeline → OK.

Step 6: Configure SCM. Under Pipeline: Definition = Pipeline script from SCM → SCM = Git → Repository URL = your GitHub HTTPS URL → Branch Specifier = `*/main` → Script Path = `Jenkinsfile` → Save.

Step 7: Run the pipeline. Click Build Now. Observe the Stage View – each stage turns green. Open the build → Console Output to read the logs.

Step 8: Check results. On the build page open Test Result (5 tests, 0 failures) and download the JAR listed under Build Artifacts. After two or more builds, a Test Result Trend graph appears on the job page.

Step 9: Automate the trigger. Job → Configure → Build Triggers → tick Poll SCM → schedule `H/2 * * * *` → Save. Push a commit and confirm a build starts automatically within two minutes.

Note: A GitHub webhook (GitHub hook trigger for GITScm polling) is faster than polling, but it requires Jenkins to be reachable from the internet. Polling is used in the lab because Jenkins runs on localhost.

Step 10: Observe a failure. Break one unit test, push, and observe that the Unit Test stage turns unstable/red, later stages are skipped and the `failure` post block runs. Fix the test and rebuild.

Expected Output:**Console Output (excerpt)**

```

[Pipeline] stage (Unit Test)
+ mvn -B test
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[Pipeline] junit
Recording test results
[Pipeline] stage (Publish Artifact)
[Pipeline] archiveArtifacts
Archiving artifacts
Build #3 of devops-demo succeeded
Finished: SUCCESS

```

Observation Table:

Build No.	Trigger	Stages passed	Tests (pass/fail)	Artifact name	Result
1	Manual (Build Now)	Checkout, Build, Unit Test, Publish Artifact (4/4)	5 pass / 0 fail	devops-demo-1.0.0-SNAPSHOT.jar	SUCCESS

Build No.	Trigger	Stages passed	Tests (pass/fail)	Artifact name	Result
2	Poll SCM (new commit)	Checkout, Build, Unit Test, Publish Artifact (4/4)	5 pass / 0 fail	devops-demo-1.0.0-SNAPSHOT.jar	SUCCESS
3	Poll SCM (broken test)	Checkout, Build (2/4); Unit Test failed	4 pass / 1 fail	None (stage skipped)	UNSTABLE / FAILURE

Exercise for Students:

1. Add a `parameters { choice(name: 'ENV', choices: ['dev', 'qa'], description: 'Target') }` block and print the selected value in a new stage.
2. Add a `stage('Docker Build')` that runs only on the main branch using `when { branch 'main' }` (attempt after Practical 6).
3. Compare the Jenkins pipeline with the GitHub Actions workflow of Practical 3 in a table (at least five points).

Observations / Output:

Paste or draw the screenshots / terminal output requested above (attach printouts if required).

Jenkins was started as a Docker container, a declarative Jenkinsfile was added to the devops-demo repository and a Pipeline job was created from SCM.

```
[Pipeline] stage (Checkout)
[Pipeline] stage (Build)
+ mvn -B -DskipTests clean package
[INFO] BUILD SUCCESS
[Pipeline] stage (Unit Test)
+ mvn -B test
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[Pipeline] junit
Recording test results
[Pipeline] stage (Publish Artifact)
[Pipeline] archiveArtifacts
Archiving artifacts
Build #3 of devops-demo succeeded
Finished: SUCCESS
```

Comparison of Jenkins and GitHub Actions (Practical 3):

1. Hosting: Jenkins is self-hosted (we ran it in Docker); GitHub Actions runs on GitHub-hosted runners.
2. Configuration file: Jenkinsfile (Groovy DSL); workflow YAML in .github/workflows.
3. Setup effort: Jenkins needs installation, plugins and maintenance; Actions needs no server setup.
4. Extensibility: Jenkins has 1800+ plugins; Actions uses reusable actions from the Marketplace.
5. Trigger: Jenkins used Poll SCM (H/2 ****) on localhost; Actions triggers instantly via GitHub events (push, pull_request).

Observation: the Test Result page showed 5 tests passed and a Test Result Trend graph appeared after several builds. In the failing build the later stages were skipped and the `post { failure }` block printed the failure message.

Result / Conclusion:

Summarise how the Jenkins pipeline automates build, test and artifact publishing.

The declarative Jenkins pipeline automated the complete flow - checkout, build, unit test and artifact publishing - with a single commit trigger. Test results were published as a trend graph and the JAR was archived as a build artifact. A failing test stopped the pipeline and prevented a faulty artifact from being published.

Questions & Answers

1. What is Jenkins? Explain the controller-agent architecture.

Ans: Jenkins is an open-source automation server used to implement CI/CD pipelines. In the controller-agent architecture, the controller (master) schedules jobs, manages configuration, plugins and the UI, while agents (nodes) are machines or containers that actually execute the build steps, which allows parallel and platform-specific builds.

2. Differentiate between declarative and scripted pipelines.

Ans: Declarative pipeline: structured syntax starting with pipeline { agent, stages, post }; simpler, validated and easier to read. Scripted pipeline: Groovy-based, starts with node { }; fully flexible but more complex and error-prone.

3. What is the purpose of the `post` section? List its conditions.

Ans: The post section defines actions that run after the stages finish, e.g., sending notifications or cleaning up. Conditions: always, success, failure, unstable, changed, fixed, regression, aborted, cleanup.

4. What do the `junit` and `archiveArtifacts` steps do?

Ans: junit reads JUnit XML test reports (target/surefire-reports/*.xml) and publishes test results and trend graphs in Jenkins. archiveArtifacts stores the specified build outputs (e.g., target/*.jar) with the build so they can be downloaded later; fingerprint: true tracks the file.

5. Explain the meaning of the cron expression `H/2 * * * *` in Jenkins.

Ans: H/2 * * * * means run about every 2 minutes. The H (hash) spreads the start time across jobs to avoid load spikes, /2 in the minute field means every 2 minutes, and the other four * mean every hour, day of month, month and day of week.

Teacher's Signature	Teacher's Note	Marks (out of 10)

Practical 5

Artifact Repository & Build Tools (Maven / Gradle + Nexus)

Aim: To study the Maven and Gradle build tools and to publish versioned build artifacts of `devops-demo` to a Sonatype Nexus Repository, and consume them as a dependency.

Course Outcome: C03 | **Duration:** 02 Hours

Objectives:

- Understand the Maven build lifecycle, GAV coordinates and SNAPSHOT vs. RELEASE versions.
- Run Nexus Repository OSS and identify hosted, proxy and group repositories.
- Deploy artifacts to Nexus using Maven (`mvn deploy`) and Gradle (`maven-publish`).

Software Required:

Sr. No.	Software / Tool	Specification	Purpose
1	Docker	24.x	Run Nexus container
2	Nexus Repository OSS	sonatype/nexus3 (needs ~2 GB RAM)	Artifact repository
3	Maven	3.9.x	Build and deploy
4	Gradle	8.x (via wrapper)	Alternative build tool

Theory:

A build tool automates compiling source code, resolving dependencies, running tests and packaging the output (JAR/WAR). **Maven** uses a declarative XML file (`pom.xml`) and a fixed lifecycle; **Gradle** uses a Groovy/Kotlin DSL (`build.gradle`) and an incremental task graph, which makes it faster for large projects.

Maven default lifecycle

validate → compile → test → package → verify → install → deploy. Running a phase runs all earlier phases. `install` copies the artifact to the local repository (`~/.m2/repository`); `deploy` uploads it to a remote repository defined in `<distributionManagement>`.

Artifact coordinates (GAV)

Every artifact is identified by `groupId : artifactId : version`, e.g., `edu.ppsu.devops:devops-demo:1.0.0`. A version ending in `-SNAPSHOT` is a work-in-progress build that can be overwritten (each upload gets a timestamp); a release version is immutable.

Artifact repository – Nexus

Repository type	Description	Default in Nexus
Hosted	Stores artifacts produced by your organisation	<code>maven-releases</code> , <code>maven-snapshots</code>
Proxy	Caches artifacts from a remote repository (Maven Central)	<code>maven-central</code>
Group	Single URL combining several repositories	<code>maven-public</code>

Feature	Maven	Gradle
Build file	<code>pom.xml</code> (XML)	<code>build.gradle</code> (Groovy) / <code>build.gradle.kts</code> (Kotlin)
Model	Fixed lifecycle & phases	Flexible task graph
Performance	Good	Faster – incremental builds, build cache, daemon
Publish command	<code>mvn deploy</code>	<code>./gradlew publish</code>

Procedure:

Part A – Run Nexus

Step 1: Start Nexus and get the admin password. Nexus takes 2–3 minutes to start.

Terminal

```
docker volume create nexus-data
docker run -d --name nexus -p 8081:8081 -v nexus-data:/nexus-data sonatype/nexus3
docker logs -f nexus          # wait for "Started Sonatype Nexus", then Ctrl+C
docker exec nexus cat /nexus-data/admin.password
```

Step 2: Initial setup. Open `http://localhost:8081` → Sign in as `admin` with the above password → set a new password → Disable anonymous access → Finish. Open Browse and note the default repositories.

Part B – Publish with Maven

Step 3: Add distribution management to `pom.xml` (inside `<project>`).

pom.xml (addition)

```
<distributionManagement>
  <repository>
    <id>nexus-releases</id>
    <url>http://localhost:8081/repository/maven-releases/</url>
  </repository>
  <snapshotRepository>
    <id>nexus-snapshots</id>
    <url>http://localhost:8081/repository/maven-snapshots/</url>
  </snapshotRepository>
</distributionManagement>
```

Step 4: Store credentials in Maven settings (never in `pom.xml`). Create / edit `~/.m2/settings.xml`:

~/.m2/settings.xml

```
<settings>
  <servers>
    <server>
      <id>nexus-releases</id>
      <username>admin</username>
      <password>YOUR_NEXUS_PASSWORD</password>
    </server>
    <server>
      <id>nexus-snapshots</id>
      <username>admin</username>
      <password>YOUR_NEXUS_PASSWORD</password>
    </server>
    <server>
      <id>nexus-public</id>
      <username>admin</username>
      <password>YOUR_NEXUS_PASSWORD</password>
    </server>
  </servers>
  <mirrors>
    <mirror>
      <id>nexus-public</id>
      <mirrorOf>*</mirrorOf>
      <url>http://localhost:8081/repository/maven-public/</url>
    </mirror>
  </mirrors>
</settings>
```

```

    </mirror>
  </mirrors>
</settings>

```

Note: The `<id>` in `settings.xml` must exactly match the `<id>` in `distributionManagement / mirror`. The mirror makes Maven download all dependencies through Nexus, so the `maven-central` proxy now caches them.

Step 5: Deploy a SNAPSHOT.

Terminal

```

mvn -B clean deploy
# Uploaded to nexus-snapshots: ../devops-demo/1.0.0-SNAPSHOT/devops-demo-1.0.0-<timestamp>-1.jar

```

Step 6: Verify in Nexus. Browse → `maven-snapshots` → `edu/ppsu/devops/devops-demo/1.0.0-SNAPSHOT/`. Run `mvn deploy` again and observe that a second timestamped file is added. Also browse `maven-central` to see cached dependencies (JUnit).

Step 7: Deploy a RELEASE.

Terminal

```

mvn versions:set -DnewVersion=1.0.0 -DgenerateBackupPoms=false
mvn -B clean deploy          # goes to maven-releases
mvn -B deploy                # run again -> fails: 400 / "does not allow updating assets"

```

Step 8: Consume the artifact. Download the released artifact from Nexus into the local repository:

Terminal

```

mvn dependency:get -Dartifact=edu.ppsu.devops:devops-demo:1.0.0
ls ~/.m2/repository/edu/ppsu/devops/devops-demo/1.0.0/

```

Part C – Same project with Gradle

Step 9: Create the Gradle build files in the project root (keep the same `src/` folder).

settings.gradle

```
rootProject.name = 'devops-demo'
```

build.gradle

```

plugins {
    id 'java'
    id 'maven-publish'
}

group = 'edu.ppsu.devops'
version = '1.1.0-SNAPSHOT'

java {
    toolchain { languageVersion = JavaLanguageVersion.of(17) }
}

repositories {
    mavenCentral()
}

dependencies {
    testImplementation 'org.junit.jupiter:junit-jupiter:5.10.2'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

test {
    useJUnitPlatform()
}

publishing {
    publications {
        mavenJava(MavenPublication) {
            from components.java
        }
    }
}

```

```

    }
    repositories {
        maven {
            name = 'nexus'
            def releasesUrl = 'http://localhost:8081/repository/maven-releases/'
            def snapshotsUrl = 'http://localhost:8081/repository/maven-snapshots/'
            url = version.endsWith('SNAPSHOT') ? snapshotsUrl : releasesUrl
            allowInsecureProtocol = true
            credentials {
                username = findProperty('nexusUser')
                password = findProperty('nexusPass')
            }
        }
    }
}

```

Step 10: Build and publish with Gradle.

Terminal

```

gradle wrapper          # generates ./gradlew (one time)
./gradlew clean build    # compile + test + jar -> build/libs/
./gradlew publish -PnexusUser=admin -PnexusPass=YOUR_NEXUS_PASSWORD

```

Step 11: Compare the build time of the second `./gradlew build` (incremental, UP-TO-DATE tasks) with `mvn clean package`.

Expected Output:

Sample output

```

[INFO] --- deploy:3.1.1:deploy (default-deploy) @ devops-demo ---
Uploading to nexus-releases: http://localhost:8081/repository/maven-releases/edu/ppsu/devops/devops-
demo/1.0.0/devops-demo-1.0.0.jar
Uploaded to nexus-releases: ... (3.1 kB at 25 kB/s)
[INFO] BUILD SUCCESS

```

Observation Table:

Sr. No.	Build tool	Version deployed	Nexus repository	Upload status / remark
1	Maven	1.0.0-SNAPSHOT	maven-snapshots	Uploaded successfully (HTTP 201)
2	Maven	1.0.0	maven-releases	Uploaded successfully
3	Maven	1.0.0 (again)	maven-releases	Rejected - HTTP 400, release versions cannot be redeployed
4	Gradle	1.1.0-SNAPSHOT	maven-snapshots	Uploaded successfully via ./gradlew publish

Exercise for Students:

1. Add a Deploy to Nexus stage to the Jenkinsfile of Practical 4 that runs `mvn -B deploy -DskipTests` (store the settings file using Jenkins Config File Provider or credentials).
2. Create a new hosted repository `ppsu-releases` in Nexus with Deployment policy = Allow redeploy and test redeploying the same version.
3. Record the time taken by `mvn clean package` and two consecutive `./gradlew build` runs.

Observations / Output:

Paste or draw the screenshots / terminal output requested above (attach printouts if required).

Nexus Repository OSS was started in Docker on port 8081, Maven was configured through settings.xml and pom.xml, and the artifact was deployed to Nexus; the same project was then published with Gradle.

```
$ mvn -B clean deploy
[INFO] --- deploy:3.1.1:deploy (default-deploy) @ devops-demo ---
Uploading to nexus-snapshots:
http://localhost:8081/repository/maven-snapshots/edu/ppsu/devops/devops-demo/1.0.0-SNAPSHOT/devops-demo-1.0.0-  
1.0.0-  
<timestamp>-1.jar
Uploaded to nexus-snapshots: ... (3.1 kB at 25 kB/s)
[INFO] BUILD SUCCESS

(version changed to 1.0.0)
$ mvn -B clean deploy
Uploaded to nexus-releases: .../maven-releases/edu/ppsu/devops/devops-demo/1.0.0/devops-demo-1.0.0.jar
[INFO] BUILD SUCCESS

$ mvn -B clean deploy      (second attempt, same version)
[ERROR] Failed to deploy artifacts: status code: 400, reason phrase: Repository does not allow updating
assets: maven-releases
[INFO] BUILD FAILURE

$ ./gradlew clean build publish
BUILD SUCCESSFUL in 9s
```

Observation: in Nexus (Browse → maven-snapshots / maven-releases) the uploaded JAR and POM files were visible under edu/ppsu/devops/devops-demo. The maven-central proxy repository showed the cached JUnit dependencies. A SNAPSHOT can be uploaded repeatedly (timestamped), but a release version is immutable. The second ./gradlew build finished faster because up-to-date tasks were skipped.

Result / Conclusion:

Explain the role of an artifact repository in a CI/CD pipeline using your observations.

An artifact repository (Nexus) acts as a central, versioned store for build outputs and dependencies. Build tools publish artifacts to it and pipelines or other teams download the same trusted artifact, which gives traceability, reproducible builds and faster dependency resolution through proxy caching. Release versions are immutable, which prevents accidental overwriting of released artifacts.

Questions & Answers

1. List and explain the phases of the Maven default lifecycle.

Ans: Default Maven lifecycle phases: validate (check project is correct), compile (compile source), test (run unit tests), package (create JAR/WAR), verify (run integration checks), install (copy to local ~/.m2 repository), deploy (copy to the remote repository such as Nexus). Running a phase runs all earlier phases.

2. What is the difference between a SNAPSHOT and a RELEASE version?

Ans: A SNAPSHOT (e.g., 1.0.0-SNAPSHOT) is a work-in-progress version that can be overwritten and re-deployed many times; each upload is timestamped. A RELEASE (e.g., 1.0.0) is a final, immutable version that cannot be re-deployed once published.

3. Explain hosted, proxy and group repositories in Nexus.

Ans: Hosted: stores artifacts produced by your own organisation (maven-releases, maven-snapshots). Proxy: caches artifacts from a remote repository such as Maven Central (maven-central). Group: combines several hosted and proxy repositories behind one URL (maven-public).

4. Why should credentials be kept in `settings.xml` and not in `pom.xml`?

Ans: pom.xml is committed to Git and shared with everyone, so credentials there would be exposed. settings.xml is stored in the user's own ~/.m2 folder (not in the repository), so usernames and passwords stay private and different users can use their own credentials.

5. Give any three differences between Maven and Gradle.

Ans: (1) Maven uses XML (pom.xml); Gradle uses Groovy/Kotlin DSL (build.gradle) which is shorter. (2) Gradle is generally faster because of incremental builds, build cache and the daemon; Maven rebuilds more. (3) Maven follows a fixed lifecycle with convention over configuration; Gradle is more flexible and task-based.

Teacher's Signature	Teacher's Note	Marks (out of 10)

Practical 6

Docker Basics & Docker Compose (containerize a sample app)

Aim: To learn basic Docker commands, containerize the `devops-demo` application using a multi-stage Dockerfile, push the image to a registry, and run a multi-container application using Docker Compose.

Course Outcome: CO4 | **Duration:** 04 Hours

Objectives:

- Differentiate containers from virtual machines; understand images, layers, containers, volumes, networks and registries.
- Write an optimized, secure multi-stage Dockerfile.
- Define and run a multi-service application (app + Nginx reverse proxy) with Docker Compose and scale it.

Software Required:

Sr. No.	Software / Tool	Specification	Purpose
1	Docker Desktop / Engine	24.x with Compose v2	Container runtime
2	Docker Hub account	Free	Public image registry
3	Base images	<code>maven:3.9-eclipse-temurin-17</code> , <code>eclipse-temurin:17-jre-alpine</code> , <code>nginx:1.27-alpine</code>	Build / runtime

Theory:

A container packages an application with all its libraries and configuration and runs it as an isolated process on the host operating system kernel. Unlike a virtual machine, which virtualizes hardware and runs a full guest OS, containers share the host kernel and therefore start in seconds and use far less memory. This solves the "works on my machine" problem by making environments identical from developer laptop to production.

Concept	Description
Image	Read-only template built in layers; each Dockerfile instruction creates a layer that is cached.
Container	A running (or stopped) instance of an image with a thin writable layer.
Registry	Stores and distributes images, e.g., Docker Hub, GitHub Container Registry (GHCR), Nexus.
Volume	Persistent storage managed by Docker that survives container removal.
Network	Virtual network; containers on the same user-defined network reach each other by name.
Docker Compose	Tool to define and run multi-container apps from a single YAML file.

Important Dockerfile instructions

`FROM` (base image), `WORKDIR` (working directory), `COPY` (copy files), `RUN` (execute at build time), `ENV` (environment variable), `EXPOSE` (document port), `USER` (run as non-root), `HEALTHCHECK` (container health), `ENTRYPOINT` / `CMD` (command executed when the container starts). A **multi-stage build**

compiles the code in a large "builder" image and copies only the final JAR into a small runtime image, reducing image size and attack surface.

Procedure:

Part A – Docker basics

Step 1: Verify Docker and run your first containers.

Terminal

```
docker version
docker run hello-world
docker run -d --name web -p 8088:80 nginx:1.27-alpine    # open http://localhost:8088
docker ps                                              # running containers
docker logs web
docker exec -it web sh                                # shell inside container; type exit to leave
docker stop web && docker rm web
docker images                                         # local images
docker ps -a                                          # all containers including stopped
```

Part B – Containerize devops-demo

Step 2: Create a `.dockerignore` so unnecessary files are not sent to the build context.

.dockerignore

```
target/
build/
.git/
.github/
*.md
```

Step 3: Create a multi-stage `Dockerfile`.

Dockerfile

```
# ----- Stage 1: build -----
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /src
COPY pom.xml .
RUN mvn -B -q dependency:go-offline          # cache dependencies as a layer
COPY src ./src
RUN mvn -B -q package -DskipTests

# ----- Stage 2: runtime -----
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
RUN addgroup -S app && adduser -S app -G app
COPY --from=build /src/target/devops-demo-*.jar app.jar
USER app
ENV APP_VERSION=1.0
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD wget -qO- http://localhost:8080/health || exit 1
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Step 4: Build, inspect and run the image.

Terminal

```
docker build -t devops-demo:1.0 .
docker images devops-demo
docker history devops-demo:1.0                # view layers
docker run -d --name demo -p 8080:8080 devops-demo:1.0
curl http://localhost:8080/                   # Hello from DevOps Demo v1.0 (host: <container-id>)
curl http://localhost:8080/health
docker ps                                     # STATUS shows (healthy) after ~30 s
docker logs demo
docker rm -f demo
```

Step 5: Push the image to Docker Hub.**Terminal**

```
docker login
docker tag devops-demo:1.0 <dockerhub-username>/devops-demo:1.0
docker push <dockerhub-username>/devops-demo:1.0
```

Part C – Docker Compose (app + Nginx reverse proxy)**Step 6:** Create the Nginx configuration.**nginx.conf**

```
server {
    listen 80;
    location / {
        proxy_pass http://app:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Step 7: Create `docker-compose.yml`.**docker-compose.yml**

```
services:
  app:
    build: .
    image: devops-demo:1.0
    environment:
      - APP_VERSION=1.0
    expose:
      - "8080"
    restart: unless-stopped

  web:
    image: nginx:1.27-alpine
    ports:
      - "8000:80"
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - app
    restart: unless-stopped
```

Step 8: Run, scale and observe.**Terminal**

```
docker compose up -d --build
docker compose ps
curl http://localhost:8000/
docker compose logs -f app           # Ctrl+C to stop following
docker compose up -d --scale app=3   # three app containers
docker compose restart web           # Nginx re-resolves 'app' to all 3 IPs
for i in 1 2 3 4 5 6; do curl -s http://localhost:8000/; done # host value changes
docker network ls                    # project network created by Compose
docker compose down                  # stop and remove everything
```

Note: In PowerShell use `1..6 | % { curl.exe -s http://localhost:8000/ }` instead of the `for` loop.

Expected Output:**Sample output (load-balanced)**

```
Hello from DevOps Demo v1.0 (host: 4f1c2a9b7d3e)
Hello from DevOps Demo v1.0 (host: 9a7e3c1d0b52)
Hello from DevOps Demo v1.0 (host: c3d8e1f2a640)
```

Observation Table:

Sr. No.	Image / Container	Size / Status	Port mapping	Remark
1	devops-demo:1.0	~190 MB (JRE-based runtime image); Running (healthy)	8000:8080	Multi-stage build; app responds on http://localhost:8000
2	build stage image (maven)	~650 MB (JDK + Maven, discarded)	-	Used only to build the JAR; not shipped
3	nginx:1.27-alpine	~50 MB; Up	8000:80	Reverse proxy and load balancer for app
4	app replicas (scaled)	3 containers; all Up	Internal 8080 (via nginx 8000)	docker compose up -d --scale app=3

Exercise for Students:

1. Build a single-stage image using `maven:3.9-eclipse-temurin-17` only and compare its size with the multi-stage image.
2. Add a `redis:7-alpine` service to the Compose file with a named volume and verify the data survives `docker compose down / up` (use `docker compose down` without `-v`).
3. Change `APP_VERSION` to `1.1`, rebuild, push tag `1.1` to Docker Hub (this tag is used in Practical 7).

Observations / Output:

Paste or draw the screenshots / terminal output requested above (attach printouts if required).

The devops-demo application was containerized using a multi-stage Dockerfile and run with Docker Compose behind an Nginx reverse proxy.

```
$ docker run hello-world
Hello from Docker!
$ docker build -t devops-demo:1.0 .
[+] Building 48.2s (14/14) FINISHED
$ docker run -d --name demo -p 8000:8080 devops-demo:1.0
$ curl http://localhost:8000/health
OK
$ docker images devops-demo
REPOSITORY    TAG       IMAGE ID      CREATED        SIZE
devops-demo   1.0       5e1a9c3b7d20 30 seconds ago 191MB

$ docker compose up -d --scale app=3
[+] Running 4/4
Container devops-demo-app-1   Started
Container devops-demo-app-2   Started
Container devops-demo-app-3   Started
Container devops-demo-nginx-1 Started
$ for i in 1 2 3; do curl -s http://localhost:8000/; done
Hello from DevOps Demo v1.0 (host: 4f1c2a9b7d3e)
Hello from DevOps Demo v1.0 (host: 9a7e3c1d0b52)
Hello from DevOps Demo v1.0 (host: c3d8e1f2a640)
```

Observation: the final image is much smaller than the build stage because it contains only the JRE and the JAR. The three requests were answered by three different container hostnames, showing that Nginx load-balances across the scaled replicas.

Result / Conclusion:

State the advantages of containerization and multi-stage builds based on the image sizes observed.

The application was packaged into a portable Docker image that runs identically on any machine. The multi-stage build produced a small runtime image (about 190 MB against about 650 MB for the build stage), reducing size and attack surface. Docker Compose started the app and Nginx together and load-balanced requests across three replicas with one command.

Questions & Answers

1. Differentiate between a container and a virtual machine.

Ans: A container shares the host operating-system kernel and isolates only the application and its dependencies, so it is lightweight (MBs), starts in seconds and uses fewer resources. A virtual machine runs a complete guest OS on a hypervisor, so it is heavy (GBs), slower to start but gives stronger isolation.

2. What is a Docker image layer? How does layer caching speed up builds?

Ans: An image is built from layers; each Dockerfile instruction (FROM, COPY, RUN) creates a read-only layer. Docker caches layers, so if an instruction and everything before it are unchanged, the cached layer is reused and only the changed layers are rebuilt - for example, copying pom.xml and downloading dependencies before copying source code makes rebuilds fast.

3. What is a multi-stage build? Why is it useful?

Ans: A multi-stage build uses more than one FROM in a Dockerfile: the first stage (with JDK and Maven) builds the JAR, and the final stage copies only the JAR into a small JRE image (COPY --from=build). It produces smaller, more secure images without build tools.

4. Differentiate between `CMD` and `ENTRYPOINT`, and between `EXPOSE` and `-p`.

Ans: `ENTRYPOINT` defines the fixed executable of the container, while `CMD` gives default arguments that can be overridden at docker run. `EXPOSE` only documents the port the container listens on; `-p` (publish) actually maps a host port to a container port (e.g., `-p 8000:8080`).

5. What is Docker Compose? How do services in Compose communicate with each other?

Ans: Docker Compose is a tool to define and run multi-container applications using a docker-compose.yml file with one command (docker compose up). Services in the same Compose project join a common network and reach each other by service name (e.g., Nginx proxies to http://app:8080) through Docker's built-in DNS.

Teacher's Signature	Teacher's Note	Marks (out of 10)

Practical 7

Kubernetes: Deploy App, Service and Scaling (minikube / kind)

Aim: To create a local Kubernetes cluster using minikube or kind, deploy the containerized `devops-demo` application using a Deployment, expose it through a Service, and perform scaling, self-healing, rolling update and rollback.

Course Outcome: C04 | **Duration:** 02 Hours

Objectives:

- Understand Kubernetes architecture and core objects: Pod, ReplicaSet, Deployment, Service.
- Write declarative YAML manifests with probes and resource limits and apply them with `kubectl`.
- Scale the application manually, observe self-healing and perform a rolling update with rollback.

Software Required:

Sr. No.	Software / Tool	Specification	Purpose
1	Docker	24.x	Container runtime / driver
2	minikube or kind	minikube 1.32+ / kind 0.22+	Local single-node cluster
3	kubectl	1.29+	Kubernetes CLI
4	Image	<code>devops-demo:1.0</code> (and <code>1.1</code>) from Practical 6	Application to deploy

Theory:

Kubernetes (K8s) is an open-source container orchestration platform that automates deployment, scaling, load-balancing and self-healing of containerized applications. You describe the desired state in YAML; Kubernetes controllers continuously work to make the actual state match it (reconciliation loop).

Architecture

Component	Runs on	Role
kube-apiserver	Control plane	Front door of the cluster; all <code>kubectl</code> commands talk to it
etcd	Control plane	Key-value store holding the entire cluster state
kube-scheduler	Control plane	Chooses a node for each new Pod
controller-manager	Control plane	Runs controllers (Deployment, ReplicaSet, Node ...)
kubelet	Every node	Starts and monitors containers of Pods on that node
kube-proxy	Every node	Implements Service networking / load-balancing
Container runtime	Every node	containerd / CRI-O – actually runs containers

Core objects

Object	Description
Pod	Smallest deployable unit; one or more containers sharing network and storage.
ReplicaSet	Keeps the specified number of identical Pods running.
Deployment	Manages ReplicaSets; provides rolling updates and rollbacks.

Object	Description
Service	Stable virtual IP / DNS name that load-balances to Pods selected by labels. Types: ClusterIP (internal), NodePort (node IP + port 30000–32767), LoadBalancer (cloud LB).
Probes	Liveness – restart container if unhealthy; Readiness – send traffic only when ready.

Procedure:

Step 1: Create a cluster (choose one option).

Terminal

```
# Option A: minikube
minikube start --driver=docker
minikube status

# Option B: kind
kind create cluster --name devops-lab

kubectl cluster-info
kubectl get nodes -o wide
```

Step 2: Make the image available inside the cluster.

Terminal

```
# minikube
minikube image load devops-demo:1.0
minikube image load devops-demo:1.1

# kind
kind load docker-image devops-demo:1.0 --name devops-lab
kind load docker-image devops-demo:1.1 --name devops-lab
```

Note: Alternatively set `image: <dockerhub-username>/devops-demo:1.0` in the manifest; the cluster will then pull it from Docker Hub.

Step 3: Write the manifests in a folder `k8s/`.

k8s/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: devops-demo
  labels:
    app: devops-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: devops-demo
  template:
    metadata:
      labels:
        app: devops-demo
    spec:
      containers:
        - name: devops-demo
          image: devops-demo:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8080
          env:
            - name: APP_VERSION
              value: "1.0"
          readinessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 5
```

```
    periodSeconds: 5
  livenessProbe:
    httpGet:
      path: /health
      port: 8080
    initialDelaySeconds: 10
    periodSeconds: 10
  resources:
    requests:
      cpu: "100m"
      memory: "128Mi"
    limits:
      cpu: "500m"
      memory: "256Mi"
```

k8s/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: devops-demo-svc
spec:
  type: NodePort
  selector:
    app: devops-demo
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30080
```

Step 4: Deploy and inspect.**Terminal**

```
kubectl apply -f k8s/
kubectl get deployments,replicasets,pods,services
kubectl describe pod <pod-name>          # events, probes, node, IP
kubectl logs <pod-name>
```

Step 5: Access the application.**Terminal**

```
# minikube
minikube service devops-demo-svc --url      # prints e.g. http://192.168.49.2:30080

# kind (or minikube) - port forwarding
kubectl port-forward svc/devops-demo-svc 8000:80
curl http://localhost:8000/
```

Step 6: Scale the Deployment.**Terminal**

```
kubectl scale deployment devops-demo --replicas=5
kubectl get pods -o wide -w          # watch new pods start (Ctrl+C)
kubectl get deployment devops-demo
```

Step 7: Observe self-healing. Delete a pod and watch Kubernetes recreate it to maintain 5 replicas.**Terminal**

```
kubectl delete pod <any-pod-name>
kubectl get pods
```

Step 8: Rolling update and rollback.**Terminal**

```
kubectl set image deployment/devops-demo devops-demo=devops-demo:1.1
kubectl rollout status deployment/devops-demo
kubectl rollout history deployment/devops-demo
curl http://localhost:8000/          # now shows v1.1 (restart port-forward if needed)
kubectl rollout undo deployment/devops-demo
kubectl rollout status deployment/devops-demo
```

Step 9: Autoscaling (optional, minikube).**Terminal**

```
minikube addons enable metrics-server
kubectl autoscale deployment devops-demo --cpu-percent=50 --min=2 --max=6
kubectl get hpa
```

Step 10: Clean up.**Terminal**

```
kubectl delete -f k8s/
minikube stop # or: kind delete cluster --name devops-lab
```

Expected Output:**Sample output**

```
$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
devops-demo-6d9f7c8b5d-2xkqp       1/1     Running   0           40s
devops-demo-6d9f7c8b5d-7hj2m       1/1     Running   0           40s
devops-demo-6d9f7c8b5d-b8wqz       1/1     Running   0           12s
devops-demo-6d9f7c8b5d-lk4tn       1/1     Running   0           12s
devops-demo-6d9f7c8b5d-r5zcv       1/1     Running   0           12s
```

Observation Table:

Sr. No.	Action	Command used	Pods (desired / ready)	Image version
1	Initial deploy	kubectl apply -f k8s/	2 / 2	devops-demo:1.0
2	Scale to 5	kubectl scale deployment devops-demo --replicas=5	5 / 5	devops-demo:1.0
3	Delete one pod	kubectl delete pod <pod-name>	5 / 5 (new pod created automatically)	devops-demo:1.0
4	Rolling update	kubectl set image deployment/devops-demo devops-demo=devops-demo:1.1	5 / 5	devops-demo:1.1
5	Rollback	kubectl rollout undo deployment/devops-demo	5 / 5	devops-demo:1.0

Exercise for Students:

1. Change the Service type to `ClusterIP`, run a temporary pod (`kubectl run tmp --rm -it --image=busybox -- sh`) and access the app using `wget -qO- http://devops-demo-svc`.
2. Set an invalid image tag (`devops-demo:9.9`), observe the `ErrImagePull` / `ImagePullBackOff` status, then roll back.
3. Create a ConfigMap holding `APP_VERSION` and refer to it from the Deployment using `valueFrom.configMapKeyRef`.

Observations / Output:

Paste or draw the screenshots / terminal output requested above (attach printouts if required).

A local cluster was created, the devops-demo Deployment (2 replicas with probes) and a NodePort Service were applied, and scaling, self-healing, rolling update and rollback were performed.

```
$ minikube start --driver=docker
$ kubectl get nodes
NAME          STATUS    ROLES          AGE    VERSION
minikube      Ready     control-plane  1m     v1.30.0
$ kubectl apply -f k8s/
deployment.apps/devops-demo created
service/devops-demo-svc created
$ kubectl scale deployment devops-demo --replicas=5
$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
devops-demo-6d9f7c8b5d-2xkqp       1/1     Running   0           40s
devops-demo-6d9f7c8b5d-7hj2m       1/1     Running   0           40s
devops-demo-6d9f7c8b5d-b8wqz       1/1     Running   0           12s
devops-demo-6d9f7c8b5d-lk4tn       1/1     Running   0           12s
devops-demo-6d9f7c8b5d-r5zcv       1/1     Running   0           12s
$ kubectl set image deployment/devops-demo devops-demo=devops-demo:1.1
$ kubectl rollout status deployment/devops-demo
deployment "devops-demo" successfully rolled out
$ curl http://localhost:8000/
Hello from DevOps Demo v1.1 (host: devops-demo-7c5b8d9f46-xk2pq)
$ kubectl rollout undo deployment/devops-demo
deployment.apps/devops-demo rolled back
```

Observation: after one pod was deleted, the ReplicaSet created a replacement within seconds and the count returned to 5/5. During the rolling update old pods were replaced gradually with no downtime, and the rollback restored version 1.0.

Result / Conclusion:

Explain how Kubernetes maintained the desired state during scaling, pod deletion and rollout.

Kubernetes continuously compared the desired state (replica count and image version) with the actual state and corrected any difference. When pods were scaled or deleted the ReplicaSet restored the required number, and the Deployment performed a rolling update and rollback without downtime. This demonstrates scaling, self-healing and zero-downtime release capability of Kubernetes.

Questions & Answers

1. Explain the Kubernetes control plane and node components.

Ans: Control plane: kube-apiserver (front end/API), etcd (cluster state store), kube-scheduler (assigns pods to nodes), kube-controller-manager (runs controllers such as ReplicaSet). Node components: kubelet (runs and monitors pods), kube-proxy (networking/service rules) and the container runtime (e.g., containerd).

2. Differentiate between a Pod, a ReplicaSet and a Deployment.

Ans: Pod: the smallest deployable unit, one or more containers sharing network and storage. ReplicaSet: keeps a specified number of identical pod replicas running. Deployment: manages ReplicaSets and provides declarative updates, rolling updates and rollbacks.

3. What are the types of Kubernetes Services? When is each used?

Ans: ClusterIP (default): internal access only inside the cluster. NodePort: exposes the service on each node's IP at a port 30000-32767 (used in local labs such as minikube). LoadBalancer: provisions an external cloud load balancer for internet access. (ExternalName maps to a DNS name.)

4. What is the difference between liveness and readiness probes?

Ans: Liveness probe checks whether the container is still healthy; if it fails, Kubernetes restarts the container. Readiness probe checks whether the container is ready to serve traffic; if it fails, the pod is removed from the Service endpoints but not restarted.

5. How does a rolling update work? How can it be rolled back?

Ans: On changing the image, the Deployment creates a new ReplicaSet and gradually scales it up while scaling the old one down (controlled by maxSurge and maxUnavailable), so the app stays available. It can be rolled back with `kubectl rollout undo deployment/<name>` (or `--to-revision=N`) and checked with `kubectl rollout history`.

Teacher's Signature	Teacher's Note	Marks (out of 10)

Practical 8

Monitoring & Logging (Prometheus + Grafana dashboard, basic alert)

Aim: To monitor the `devops-demo` application and its host using Prometheus, visualize metrics on a Grafana dashboard, configure a basic alert rule, and inspect application logs.

Course Outcome: C05 | **Duration:** 02 Hours

Objectives:

- Understand monitoring vs. observability and the three pillars: metrics, logs and traces.
- Configure Prometheus to scrape application and node metrics; write basic PromQL queries.
- Build a Grafana dashboard and trigger an alert; view and filter container logs.

Software Required:

Sr. No.	Software / Tool	Specification	Purpose
1	Docker	Compose v2	Run the monitoring stack
2	Prometheus	<code>prom/prometheus:v2.54.1</code>	Metrics collection, alert rules
3	Node Exporter	<code>prom/node-exporter:v1.8.2</code>	Host (CPU, memory, disk) metrics
4	Grafana	<code>grafana/grafana:11.2.0</code>	Dashboards and visualization

Theory:

Monitoring tells us whether a system is working by tracking known metrics against thresholds.

Observability is the ability to understand why it is not working by asking new questions of the telemetry the system emits. Observability rests on three pillars: **metrics** (numeric time series), **logs** (timestamped event records) and **traces** (the path of a single request across services).

Prometheus is an open-source monitoring system that pulls (scrapes) metrics over HTTP from `/metrics` endpoints at regular intervals and stores them in a time-series database. Metrics are queried using PromQL. Exporters (e.g., Node Exporter) expose metrics for systems that do not do so natively. Alert rules are evaluated by Prometheus; firing alerts are sent to Alertmanager for routing to e-mail, Slack, etc. **Grafana** connects to Prometheus (and many other sources) to build interactive dashboards.

Metric type	Meaning	Example from devops-demo
Counter	Only increases (reset on restart)	<code>http_requests_total</code>
Gauge	Goes up and down	<code>jvm_memory_used_bytes</code> , <code>app_uptime_seconds</code>
Histogram / Summary	Distribution of values (latency)	— (exercise)

Useful PromQL queries

Query	Meaning
<code>up</code>	1 if target scraped successfully, else 0
<code>rate(http_requests_total[1m])</code>	Per-second request rate (per path) over last minute
<code>sum(rate(http_requests_total[1m]))</code>	Total request rate of the app
<code>jvm_memory_used_bytes / 1024 / 1024</code>	Heap memory in MB

Query	Meaning
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)	Host CPU usage %

Golden signals

Google SRE recommends monitoring four golden signals for every service: **Latency, Traffic, Errors and Saturation**. DevOps teams also track delivery KPIs such as Lead time for changes and MTTR (Mean Time To Recover).

Procedure:

Step 1: Create the folder structure in the `devops-demo` repository: `monitoring/prometheus.yml` and `monitoring/alert.rules.yml`.

monitoring/prometheus.yml

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  - /etc/prometheus/alert.rules.yml

scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets: ['localhost:9090']

  - job_name: devops-demo
    metrics_path: /metrics
    static_configs:
      - targets: ['app:8080']

  - job_name: node
    static_configs:
      - targets: ['node-exporter:9100']
```

monitoring/alert.rules.yml

```
groups:
  - name: devops-demo-alerts
    rules:
      - alert: AppDown
        expr: up{job="devops-demo"} == 0
        for: 1m
        labels:
          severity: critical
        annotations:
          summary: "devops-demo is down"
          description: "Target {{ $labels.instance }} unreachable for more than 1 minute."

      - alert: HighRequestRate
        expr: sum(rate(http_requests_total{job="devops-demo"}[1m])) > 2
        for: 1m
        labels:
          severity: warning
        annotations:
          summary: "High request rate on devops-demo"
          description: "More than 2 requests/second for 1 minute."
```

Step 2: Create the monitoring Compose file.

docker-compose.monitoring.yml

```
services:
  app:
    build: .
    image: devops-demo:1.0
```

```

ports:
  - "8000:8080"

prometheus:
  image: prom/prometheus:v2.54.1
  ports:
    - "9090:9090"
  volumes:
    - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    - ./monitoring/alert.rules.yml:/etc/prometheus/alert.rules.yml:ro

node-exporter:
  image: prom/node-exporter:v1.8.2
  ports:
    - "9100:9100"

grafana:
  image: grafana/grafana:11.2.0
  ports:
    - "3000:3000"
  environment:
    - GF_SECURITY_ADMIN_PASSWORD=admin
  volumes:
    - grafana-data:/var/lib/grafana
  depends_on:
    - prometheus

volumes:
  grafana-data:

```

Step 3: Start the stack.

Terminal

```

docker compose -f docker-compose.monitoring.yml up -d --build
docker compose -f docker-compose.monitoring.yml ps
curl http://localhost:8000/metrics

```

Step 4: Explore Prometheus. Open <http://localhost:9090> → Status → Targets: all three targets should be UP. In the Graph tab execute the PromQL queries given in the theory. Open Alerts to see both rules in inactive state.

Step 5: Generate traffic.

Terminal

```

# Bash / Git Bash
for i in $(seq 1 1500); do curl -s http://localhost:8000/ > /dev/null; sleep 0.1; done

# PowerShell
1..1500 | % { curl.exe -s http://localhost:8000/ > $null; Start-Sleep -Milliseconds 100 }

```

Step 6: Configure Grafana. Open <http://localhost:3000> (admin / admin) → Connections → Data sources → Add data source → Prometheus → URL <http://prometheus:9090> → Save & test.

Step 7: Build a dashboard (Dashboards → New → New dashboard → Add visualization) with four panels:

Panel	Visualization	PromQL
App status	Stat	<code>up{job="devops-demo"}</code>
Requests / sec	Time series	<code>sum by (path) (rate(http_requests_total[1m]))</code>
Heap memory (MB)	Gauge	<code>jvm_memory_used_bytes / 1024 / 1024</code>
Host CPU %	Time series	<code>100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)</code>

Save the dashboard as `devops-demo Overview`. Optionally import the community Node Exporter Full dashboard (ID 1860) through Dashboards → New → Import.

Step 8: Trigger the alerts. While the traffic loop of Step 5 runs, the `HighRequestRate` alert moves Pending → Firing after 1 minute. Then stop the application and watch `AppDown` fire:

Terminal

```
docker compose -f docker-compose.monitoring.yml stop app
# after ~1-2 minutes check http://localhost:9090/alerts -> AppDown FIRING
docker compose -f docker-compose.monitoring.yml start app      # alert resolves
```

Step 9: Logging. The application writes one log line per request to standard output, which Docker collects.

Terminal

```
docker compose -f docker-compose.monitoring.yml logs app --tail 20
docker compose -f docker-compose.monitoring.yml logs -f app | grep " 400"      # only errors
docker compose -f docker-compose.monitoring.yml logs --since 5m prometheus
```

Note: In production, logs are shipped to a central store such as the ELK stack (Elasticsearch, Logstash, Kibana) or Grafana Loki so they can be searched alongside metrics.

Step 10: Clean up: `docker compose -f docker-compose.monitoring.yml down`

Expected Output:

Prometheus Alerts page (sample)

```
Alerts
devops-demo-alerts
  AppDown      (1 active)  FIRING      severity="critical"
  HighRequestRate (0 active)  INACTIVE
```

Observation Table:

Sr. No.	PromQL query / panel	Observed value	Alert state	Time
1	<code>up{job="devops-demo"}</code> (App status)	1 (UP)	AppDown: Inactive	10:05
2	<code>sum by (path) (rate(http_requests_total[1m]))</code>	~10 req/s for path "/" during the traffic loop	HighRequestRate: Firing	10:12
3	<code>jvm_memory_used_bytes / 1024 / 1024</code> (Heap memory)	~38 MB	Inactive	10:14
4	<code>up{job="devops-demo"}</code> after stopping the app	0 (DOWN)	AppDown: Firing	10:20

Exercise for Students:

1. Add an Alertmanager container and route alerts to an e-mail or webhook receiver.
2. Create a Grafana alert rule (instead of a Prometheus rule) that fires when heap memory exceeds 100 MB.
3. Add a request-duration metric to `App.java` and plot the average latency in Grafana.

Observations / Output:

Paste or draw the screenshots / terminal output requested above (attach printouts if required).

Prometheus, Node Exporter and Grafana were started with `docker-compose.monitoring.yml`. All three Prometheus targets were UP, the Grafana dashboard 'devops-demo Overview' was built with four panels and both alert rules were tested.

```
$ docker compose -f docker-compose.monitoring.yml up -d --build
$ curl http://localhost:8000/metrics
# TYPE http_requests_total counter
http_requests_total{path="/"} 1500
# TYPE jvm_memory_used_bytes gauge
jvm_memory_used_bytes 3.9845888E7

Prometheus -> Status -> Targets:
prometheus (1/1 up), devops-demo (1/1 up), node (1/1 up)

$ docker compose -f docker-compose.monitoring.yml stop app
(after ~1-2 min) http://localhost:9090/alerts
AppDown (1 active)          FIRING severity="critical"
HighRequestRate (0 active)  INACTIVE

$ docker compose -f docker-compose.monitoring.yml logs app --tail 3
app-1 | GET / 200
app-1 | GET /health 200
app-1 | GET /add?a=2&b=3 200
```

Observation: while the curl loop ran, HighRequestRate moved from Pending to Firing after one minute. Stopping the app made up become 0 and AppDown fired after the 1-minute 'for' period; it resolved after the app was restarted.

Result / Conclusion:

Describe how metrics, dashboards, alerts and logs together help detect and diagnose failures.

Metrics (Prometheus), dashboards (Grafana), alerts and logs together give complete visibility of the system. The dashboard showed the current health and load, the alert notified us within about a minute when the application stopped, and the logs helped locate what happened. This reduces the time to detect and diagnose failures (lower MTTR).

Questions & Answers

1. Differentiate between monitoring and observability. What are the three pillars of observability?

Ans: Monitoring tells you that something is wrong by tracking known metrics and alerts; observability lets you understand why it is wrong by exploring the system's internal state through its outputs, even for unknown problems. The three pillars of observability are metrics, logs and traces.

2. Explain the pull model of Prometheus. What is an exporter?

Ans: In the pull model Prometheus periodically scrapes (HTTP GET) the /metrics endpoint of each target as defined in prometheus.yml, instead of targets pushing data. An exporter is a program that exposes metrics of a system in Prometheus format - e.g., Node Exporter exposes CPU, memory and disk metrics of the host.

3. Differentiate between a counter and a gauge with examples.

Ans: A counter only increases (or resets to zero on restart) and counts events, e.g., http_requests_total; it is usually used with rate(). A gauge can go up and down and shows the current value, e.g., jvm_memory_used_bytes or temperature.

4. What is the purpose of the `for` clause in a Prometheus alert rule?

Ans: The 'for' clause makes the alert condition stay true for the given duration (e.g., 1m) before the alert goes from Pending to Firing. It avoids false alarms from short, temporary spikes.

5. What are the four golden signals? Define MTTR.

Ans: The four golden signals are latency, traffic, errors and saturation. MTTR (Mean Time To Recovery/Restore) is the average time taken to restore service after a failure or incident, from detection to recovery.

Teacher's Signature	Teacher's Note	Marks (out of 10)

Practical 9

Mini DevOps Project (end-to-end: Git → CI → CD → Monitor)

Aim: To design and implement a complete DevOps pipeline for an application in a team, using Agile (Scrum/Kanban) planning, Git-based collaboration, automated CI, containerized continuous deployment to Kubernetes, and monitoring with alerts — and to measure delivery performance using DevOps KPIs.

Course Outcome: CO1, CO2, CO3, CO4, CO5 | **Duration:** 08 Hours

Objectives:

- Plan work as user stories on a Kanban/Scrum board and deliver in short iterations.
- Integrate all tools from Practicals 1–8 into one automated value stream.
- Measure DORA metrics (deployment frequency, lead time, change failure rate, MTTR) and reflect on continuous improvement in a retrospective.

Team & Tools:

Teams of 3–4 students. Roles rotate each session: Product Owner, Scrum Master, Developers. The team may use `devops-demo` or any small application of its choice (Java, Python, Node.js) that has unit tests and a health endpoint.

Sr. No.	Software / Tool	Specification	Purpose
1	GitHub (Repo, Projects, Actions, GHCR)	Free	Code, board, CI/CD, image registry
2	Docker / Docker Compose	24.x	Containerization, monitoring stack
3	kind / minikube + kubectl	Latest	Kubernetes deployment target
4	Prometheus + Grafana	As in Practical 8	Monitoring and alerting
5	Jenkins / Nexus (optional)	As in Practical 4–5	Alternative CI/CD path

Theory:

DevOps aims to shorten the system development life cycle and provide continuous delivery with high quality. The **Three Ways of DevOps** are: (1) **Flow** – fast, left-to-right flow of work from Dev to Ops; (2) **Feedback** – fast, constant feedback from right to left (tests, monitoring, alerts); and (3) **Continual Learning and Experimentation** – retrospectives, blameless post-mortems and improvement.

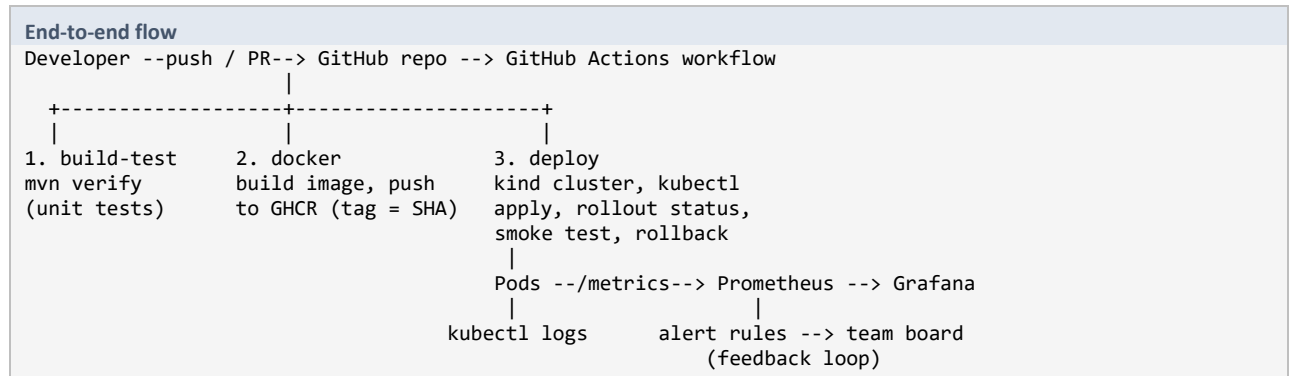
Continuous Delivery means every change that passes the automated pipeline is releasable; **Continuous Deployment** goes further and releases every such change to production automatically. Safe release techniques include rolling updates, blue/green deployments (switch traffic between two identical environments), canary releases (send a small percentage of traffic to the new version) and feature flags.

DORA metrics

Metric	Definition	How to measure in this project
Deployment Frequency	How often code is deployed	Number of successful <code>deploy</code> job runs per day
Lead Time for Changes	Commit → running in deployment	Time between commit and green <code>deploy</code> job

Metric	Definition	How to measure in this project
Change Failure Rate	% of deployments causing failure	Failed or rolled-back deploys ÷ total deploys
MTTR	Mean time to restore service	Time from AppDown alert firing to resolved

Pipeline Architecture



Procedure (session-wise plan):

Session 1 – Plan (Agile) and set up repository [2 hrs]

1. Create a GitHub Project (Board layout) with columns Backlog, To do, In progress, In review, Done. Set a WIP limit of 2 for In progress.
2. Write at least 6 user stories in the format "As a <role>, I want <goal> so that <benefit>" with acceptance criteria, e.g., "As an operator, I want an alert when the service is down so that I can restore it quickly."
3. Estimate stories using story points (1, 2, 3, 5, 8) by planning poker and select stories for Sprint 1 (Sessions 2–3).
4. Create the repository, add all team members, protect `main` (PR + 1 approval + status check) and add `CODEOWNERS`.

Session 2 – Continuous Integration and containerization [2 hrs]

1. Each member works on a feature branch and raises PRs; every PR must be reviewed by another member.
2. Add the `Dockerfile`, `.dockerignore` and `k8s/` manifests from Practicals 6 and 7.
3. Add the CI/CD workflow below at `.github/workflows/cicd.yml`. On a pull request only the `build-test` job runs; on a push to `main` all three jobs run.

```

.github/workflows/cicd.yml
name: CI-CD

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

permissions:
  contents: read
  packages: write

env:
  IMAGE: ghcr.io/${{ github.repository_owner }}/devops-demo

jobs:
  
```

```

build-test:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-java@v4
      with:
        distribution: temurin
        java-version: '17'
        cache: maven
    - name: Build and unit test
      run: mvn -B clean verify

docker:
  needs: build-test
  if: github.event_name == 'push'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${github.actor}
        password: ${secrets.GITHUB_TOKEN}
    - uses: docker/build-push-action@v6
      with:
        context: .
        push: true
        tags: |
          ${env.IMAGE}:${github.sha}
          ${env.IMAGE}:latest

deploy:
  needs: docker
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${github.actor}
        password: ${secrets.GITHUB_TOKEN}
    - name: Create kind cluster
      uses: helm/kind-action@v1
      with:
        cluster_name: ci
    - name: Load image into cluster
      run: |
        docker pull ${env.IMAGE}:${github.sha}
        kind load docker-image ${env.IMAGE}:${github.sha} --name ci
    - name: Deploy
      run: |
        sed -i "s|image: .*|image: ${env.IMAGE}:${github.sha}|" k8s/deployment.yaml
        kubectl apply -f k8s/
        kubectl rollout status deployment/devops-demo --timeout=180s
    - name: Smoke test
      run: |
        kubectl port-forward svc/devops-demo-svc 8000:80 &
        sleep 5
        curl -f http://localhost:8000/health
        curl -f "http://localhost:8000/add?a=2&b=3"
    - name: Rollback on failure
      if: failure()
      run: kubectl rollout undo deployment/devops-demo || true

```

Note: GHCR image names must be lowercase. If your GitHub username contains capital letters, replace `${github.repository_owner}` with your username in lowercase. After the first push, make the package public from GitHub profile → Packages → [devops-demo](#) → Package settings if you want to pull it without login.

Session 3 – Continuous Deployment and release strategies [2 hrs]

1. Push to `main` and verify that `build-test` → `docker` → `deploy` all turn green. Record the run time of each job (lead time).

2. Make a visible change (e.g., set `APP_VERSION` to `1.1` in `k8s/deployment.yaml`), merge through a PR and confirm the new version is deployed by the pipeline.
3. Introduce a deliberate failure (e.g., change the readiness probe path to `/healthz`) and observe the rollout timeout, the failed smoke test and the automatic rollback step. Count it in change failure rate.
4. Local CD (optional): deploy the GHCR image to your local minikube cluster, either manually with `kubectl set image` or by adding the Jenkins stage below to the Jenkinsfile of Practical 4 (Jenkins needs `kubectl` and a kubeconfig).

Jenkinsfile (optional CD stage)

```
stage('Deploy to minikube') {
  when { branch 'main' }
  steps {
    sh '''
      kubectl set image deployment/devops-demo \
        devops-demo=ghcr.io/<owner>/devops-demo:${GIT_COMMIT}
      kubectl rollout status deployment/devops-demo --timeout=120s
    '''
  }
  post {
    failure { sh 'kubectl rollout undo deployment/devops-demo' }
  }
}
```

Session 4 – Monitor, measure, retrospect and present [2 hrs]

1. Run the application locally (Compose or minikube with `kubectl port-forward`) and attach the Prometheus + Grafana stack of Practical 8. Update the `devops-demo` scrape target if the app is reached through port-forward (e.g., `host.docker.internal:8000`).
2. Build a team dashboard with: app status, request rate, memory, and a panel showing the current version (`app_info`).
3. Simulate an incident by stopping the app; record the time the alert fires and the time service is restored → calculate MTTR.
4. Fill the DORA metrics table below and hold a sprint retrospective (What went well? What did not? What will we improve?).
5. Present a 10-minute demo: board → PR → pipeline run → deployment → dashboard/alert.

Deliverables:

- GitHub repository link (code, Dockerfile, manifests, workflow, monitoring configs, README with architecture diagram).
- Screenshots: project board, a reviewed PR, successful and failed pipeline runs, `kubectl get pods`, Grafana dashboard, firing alert.
- Completed DORA metrics table and retrospective notes (below).
- Short project report (4–6 pages) and presentation slides.

Suggested Project Ideas:

- Student attendance REST API with health and metrics endpoints.
- URL shortener service with Redis (Docker Compose + Kubernetes).
- Library book-search microservice with a simple front end served by Nginx.
- To-do list web application with a canary release using two Deployments behind one Service.

Team Details:

Sr. No.	Name	Enrollment No.	Role(s)	Contribution
1	Shah Rishabh Alpeshbhai	23SS02IT181	Scrum Master, DevOps Engineer	Sprint planning and board, Git workflow and PR reviews, CI/CD pipeline (GitHub Actions), Kubernetes deployment, monitoring
2				
3				
4				

DORA Metrics Observation:

Metric	Raw data collected	Calculated value
Deployment Frequency	6 successful deployments to Kubernetes during the 2-week project period	6 deployments / 2 weeks = about 3 per week
Lead Time for Changes	Commit-to-deploy time of 3 changes: 9, 11 and 10 minutes	Average = 10 minutes
Change Failure Rate	6 deployments, 1 failed (readiness probe changed to /healthz, rolled back automatically)	1 / 6 = about 17%
MTTR	Alert fired at 10:20; service restored at 10:26 after restarting the app	6 minutes

Sprint Retrospective:

What went well	What did not go well	Action items
Pull requests with CI checks caught errors before merge. Automatic rollback restored service quickly. Grafana dashboard and alerts gave quick visibility.	Pipeline was slow on the first run because Maven dependencies were not cached. One deployment failed because of a wrong probe path. Some tasks were started late on the board.	Enable Maven dependency caching in CI. Add a smoke-test and manifest validation before deploy. Split user stories into smaller tasks and update the board daily.

Evaluation Rubric (Mini Project):

Criterion	Marks
Agile planning – user stories, estimation, board usage	2
Git workflow – branches, PRs, reviews, protected main	2
CI/CD pipeline – build, test, image, deploy, rollback working	3
Monitoring – dashboard, alert, MTTR measured	1.5

Criterion	Marks
Presentation, report and retrospective	1.5
Total	10

Exercise for Students:

1. Add a security stage to the pipeline: dependency/image scanning with `aquasecurity/trivy-action` and fail on CRITICAL vulnerabilities (SAST/DAST introduction).
2. Replace the rolling update with a blue/green deployment using two Deployments (blue, green) and switch the Service selector.
3. Write the infrastructure (kind cluster or a cloud VM) as code using Terraform and configure it with Ansible.

Observations / Output:

Paste or draw the screenshots / terminal output requested above (attach printouts if required).

Project: devops-demo end-to-end pipeline - Git → CI → CD → Monitor. Work was planned on a Kanban/Scrum board, code changes were made through reviewed pull requests, GitHub Actions built, tested, pushed the image to GHCR and deployed it to Kubernetes with smoke test and automatic rollback, and Prometheus + Grafana monitored the application.

```

build-test  PASSED  (mvn -B verify: Tests run: 5, Failures: 0)  1m 02s
docker      PASSED  (image pushed to ghcr.io/<owner>/devops-demo:<sha>)  1m 18s
deploy      PASSED  (rollout status: successfully rolled out)  0m 55s

$ kubectl get pods
NAME                                READY  STATUS   RESTARTS  AGE
devops-demo-6d9f7c8b5d-2xkqp        1/1    Running  0          2m
devops-demo-6d9f7c8b5d-7hj2m        1/1    Running  0          2m
$ curl -f http://localhost:8000/health
OK

Failed run (readiness probe path changed to /healthz):
error: deployment "devops-demo" exceeded its progress deadline
Rollback on failure -> deployment.apps/devops-demo rolled back

Incident test: app stopped -> AppDown FIRING after ~1 min -> app restarted -> RESOLVED
MTTR = 6 minutes

```

Observation: the whole path from commit to running pod was automatic and took about 10 minutes. The deliberate failure was detected by the rollout timeout and the failed smoke test, and the rollback step restored the previous working version without manual work.

Result / Conclusion:

Reflect on the complete value stream you built and the improvements identified in the retrospective.

A complete DevOps value stream was built: Agile planning → Git/PR workflow → automated CI → containerized CD on Kubernetes → monitoring and alerting. The DORA metrics (about 3 deployments per week, 10-minute lead time, 17% change failure rate and 6-minute MTTR) show how fast and stable delivery was. The retrospective identified improvements such as dependency caching, pre-deploy validation and smaller tasks for continuous improvement.

Questions & Answers

1. Explain the Three Ways of DevOps with examples from your project.

Ans: First Way - Flow: speed of work from Dev to Ops; in our project the CI/CD pipeline and small PRs made changes flow to production automatically. Second Way - Feedback: fast feedback loops; failed tests, failed rollouts and Grafana alerts told us about problems immediately. Third Way - Continual learning and experimentation: we held a retrospective, tried failure scenarios (bad probe) and improved the process.

2. Differentiate between Continuous Integration, Continuous Delivery and Continuous Deployment.

Ans: Continuous Integration: every change is automatically built and tested after merging. Continuous Delivery: the code is always kept in a deployable state and is released to production by a manual approval. Continuous Deployment: every change that passes all automated tests is deployed to production automatically with no manual step.

3. What are blue/green and canary deployments? How do they reduce risk?

Ans: Blue/green deployment keeps two identical environments - the live (blue) and the new version (green); after testing green, traffic is switched to it (e.g., by changing the Service selector) and can be switched back instantly. Canary deployment sends the new version to a small percentage of users first and increases gradually. Both reduce risk by limiting the impact of a bad release and allowing quick rollback.

4. Define the four DORA metrics and state the values obtained in your project.

Ans: Deployment Frequency: how often code is deployed to production (about 3 per week). Lead Time for Changes: time from commit to running in production (about 10 minutes). Change Failure Rate: percentage of deployments causing a failure (1 of 6, about 17%). MTTR: average time to restore service after a failure (6 minutes).

5. What is a sprint retrospective? List two improvements your team identified.

Ans: A sprint retrospective is a meeting at the end of a sprint where the team reviews what went well, what did not and what to improve. Two improvements identified: (1) enable Maven dependency caching in the CI pipeline to reduce build time; (2) add manifest validation and a smoke test before deployment to reduce failed deployments.

Teacher's Signature	Teacher's Note	Marks (out of 10)